

Relational schema quality

09/12/2026 08:40:10

<https://chatgpt.com/share/69129a39-754c-8007-a026-fa371cccd2e7>

In Chapters 3 through 6, we presented various aspects of the relational...
This passage is an introduction to Chapter 15: "Basics of Functional...
15.1 Informal Design Guidelines for Relation Schemas Before discussing...
Excellent — this section (15.1.1 Imparting Clear Semantics to Attributes...
15.1.2 Redundant Information in Tuples and Update Anomalies One goa...
Excellent — this section (15.1.2 Redundant Information in Tuples and...
NULL Values in Tuples In some schema designs we may group many...
Excellent — this passage continues the informal design guidelines for...
15.1.5 Summary and Discussion of Design Guidelines In Sections 15.1.1...
Perfect — this section (15.1.5 Summary and Discussion of Design...
15.2 Functional Dependencies So far we have dealt with the informal...
Here's a clear, structured summary and explanation of your excerpt on...
15.3.1 Normalization of Relations The normalization process, as first...
Here's a clear, structured explanation and summary of your excerpt on...
15.3.4 First Normal Form First normal form (1NF) is now considered to b...
Excellent — you've just covered Sections 15.3.4 to 15.3.6 of Elmasri &...
General Definitions of Second and Third Normal Forms In general, we...
Excellent — this section you've pasted covers the formal (general)...
Boyce-Codd Normal Form Boyce-Codd normal form (BCNF) was propose...
Excellent — this passage discusses the Boyce-Codd Normal Form (BCN...
Multivalued Dependency and Fourth Normal Form So far we have...
Excellent — this section moves beyond functional dependencies (FDs)...
dependent ??? you mean department
Good catch — but in this case, "dependent" is actually correct, not...
15.7 Join Dependencies and Fifth Normal Form In our discussion so far, ...
Excellent — this section is the final step in the normalization hierarchy,...
Demorgan's axioms

De Morgan's Axioms (or De Morgan's Laws) are two fundamental rules i...
well... what does it have to do with database design

Excellent question — and that's exactly the kind of thinking that leads t...
i mean what does demorgan's axioms have to do with database design

Ah — excellent clarification. 🙌 You're asking: why do we even care abo...



In Chapters 3 through 6, we presented various aspects of the relational model and the languages associated with it. Each relation schema consists of a number of attributes, and the relational database schema consists of a number of relation schemas. So far, we have assumed that attributes are grouped to form a relation schema by using the common sense of the database designer or by mapping a database schema design from a conceptual data model such as the ER or Enhanced-ER (EER) data model. These models make the designer identify entity types and relationship types and their respective attributes, which leads to a natural and logical grouping of the attributes into relations when the mapping procedures discussed in Chapter 9 are followed. However, we still need some formal way of analyzing why one grouping of attributes into a relation schema may be better than another. While discussing database design in Chapters 7 through 10, we did not develop any measure of appropriateness or goodness to measure the quality of the design, other than the intuition of the designer. In this chapter we discuss some of the theory that has been developed with the goal of evaluating relational schemas for design quality—that is, to measure formally why one set of groupings of attributes into relation schemas is better than another.

There are two levels at which we can discuss the goodness of relation schemas. The first is the logical (or conceptual) level—how users interpret the relation schemas and the meaning of their attributes. Having good relation schemas at this level

enables users to understand clearly the meaning of the data in the relations, and hence to formulate their queries correctly. The second is the implementation (or

15 chapter

15

502

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

physical storage) level—how the tuples in a base relation are stored and updated.

This level applies only to schemas of base relations—which will be physically stored

as files—whereas at the logical level we are interested in schemas of both base rela

tions and views (virtual relations). The relational database design theory developed

in this chapter applies mainly to base relations, although some criteria of appropri

ateness also apply to views, as shown in Section 15.1.

As with many design problems, database design may be performed using two

approaches: bottom-up or top-down. A bottom-up design methodology (also

called design by synthesis) considers the basic relationships among individual attrib

utes as the starting point and uses those to construct relation schemas. This

approach is not very popular in practice¹ because it suffers from the problem of

having to collect a large number of binary relationships among attributes as the

starting point. For practical situations, it is next to impossible to capture binary

relationships among all such pairs of attributes. In contrast, a top-down design

methodology (also called design by analysis) starts with a number of groupings of

attributes into relations that exist together naturally, for example, on an invoice, a

form, or a report. The relations are then analyzed

individually and collectively, leading to further decomposition until all desirable properties are met. The theory described in this chapter is applicable to both the top-down and bottom-up design approaches, but is more appropriate when used with the top-down approach.

Relational database design ultimately produces a set of relations. The implicit goals of the design activity are information preservation and minimum redundancy.

Information is very hard to quantify—hence we consider information preservation in terms of maintaining all concepts, including attribute types, entity types, and relationship types as well as generalization/specialization relationships, which are described using a model such as the EER model. Thus, the relational design must preserve all of these concepts, which are originally captured in the conceptual design after the conceptual to logical design mapping.

Minimizing redundancy implies minimizing redundant storage of the same information and reducing the need for multiple updates to maintain consistency across multiple copies of the same information in response to real-world events that require making an update.

We start this chapter by informally discussing some criteria for good and bad relation schemas in Section 15.1. In Section 15.2, we define the concept of functional dependency, a formal constraint among attributes that is the main tool for formally measuring the appropriateness of attribute groupings into relation schemas. In

Section 15.3, we discuss normal forms and the process of normalization using functional dependencies. Successive normal forms are defined to meet a set of desirable

constraints expressed using functional dependencies. The normalization procedure consists of applying a series of tests to relations to meet these increasingly stringent requirements and decompose the relations when necessary. In Section 15.4, we discuss more general definitions of normal forms that can be directly applied to any given design and do not require step-by-step analysis and normalization. Sections 15.5 to 15.7 discuss further normal forms up to the fifth normal form. In Section 15.6 we introduce the multivalued dependency (MVD), followed by the join dependency (JD) in Section 15.7. Section 15.8 summarizes the chapter. Chapter 16 continues the development of the theory related to the design of good relational schemas. We discuss desirable properties of relational decomposition—nonadditive join property and functional dependency preservation property. A general algorithm that tests whether or not a decomposition has the nonadditive (or lossless) join property (Algorithm 16.3 is also presented). We then discuss properties of functional dependencies and the concept of a minimal cover of dependencies. We consider the bottom-up approach to database design consisting of a set of algorithms to design relations in a desired normal form. These algorithms assume as input a given set of functional dependencies and achieve a relational design in a target

15.1 Informal Design Guidelines for Relation Schemas

503

503

Chapter 16 continues the development of the theory related to the design of good relational schemas. We discuss desirable properties of relational decomposition—nonadditive join property and functional dependency preservation property. A general algorithm that tests whether or not a decomposition has the nonadditive (or lossless) join property (Algorithm 16.3 is also presented). We then discuss properties of functional dependencies and the concept of a minimal cover of dependencies. We consider the bottom-up approach to database design consisting of a set of algorithms to design relations in a desired normal form. These algorithms assume as input a given set of functional dependencies and achieve a relational design in a target

get normal form while adhering to the above desirable properties. In Chapter 16 we also define additional types of dependencies that further enhance the evaluation of the goodness of relation schemas. If Chapter 16 is not covered in a course, we recommend a quick introduction to the desirable properties of decomposition and the discussion of Property NJB in Section 16.2.

08/11/2025, 09:15:25



This passage is an **introduction to Chapter 15: “Basics of Functional Dependencies and Normalization for Relational Databases”** — a core topic in relational database design theory.

Let’s break it down clearly and summarize the main ideas section by section 📌

◆ Purpose of the Chapter

The main goal is to **analyze and improve the quality of relational database schemas** — that is, to determine why some groupings of attributes into relations are *better* than others.

Up to this point (Chapters 3–10), database design relied mostly on **intuition** or **mapping from conceptual models** (like ER/EER diagrams).

Now, this chapter introduces **formal tools** (especially *functional dependencies* and *normalization*) to evaluate design quality.

◆ Two Levels of Schema “Goodness”

1. Logical (or conceptual) level

- Concerned with how users interpret relations and attributes.
- A *good schema* at this level is clear and meaningful — users easily understand data and write correct queries.

2. Physical (or implementation) level

- Deals with how base relation tuples are *stored and updated* in physical files.
 - Applies only to *base relations* (those physically stored), not *views*.
-

◆ Design Approaches

There are **two main approaches** to database design:

1. Bottom-Up (Design by Synthesis)

- Starts with relationships among individual attributes (for example, which attributes depend on others).
- Builds relation schemas from these dependencies.
- *Problem:* Requires collecting too many binary relationships among attributes — impractical for large systems.
- *Example:* NIAM (Natural language Information Analysis Method).

2. Top-Down (Design by Analysis)

- Starts from natural groupings (like data on forms, invoices, reports).
- Then analyzes and decomposes these groups until they meet all desirable properties.
- **Most commonly used in practice.**

The theory in this chapter applies to both, but is especially suited to the **top-down approach**.

◆ Goals of Relational Design

1. Information Preservation

- The design must retain all the concepts (entity types, attributes, relationships, generalizations, etc.) captured in the conceptual model (ER/EER).
- Nothing should be lost when converting from conceptual to logical design.

2. Minimum Redundancy

- Avoid storing the same information multiple times.

- Reduces update anomalies and keeps the database consistent.

◆ Topics Covered in This Chapter (15)

Section	Topic	Description
15.1	Informal Design Guidelines	Discusses intuitive principles for identifying good and bad relation schemas.
15.2	Functional Dependencies (FDs)	Introduces <i>functional dependency</i> — the key formal tool for measuring schema quality.
15.3	Normalization Process	Explains how to use FDs to decompose relations into better forms (1NF → 2NF → 3NF → BCNF, etc.).
15.4-15.7	Advanced Normal Forms	Discusses higher normal forms — 4NF, 5NF — using more advanced dependencies like multivalued (MVD) and join dependencies (JD).
15.8	Summary	Wraps up key points of the chapter.

◆ What Comes Next (Chapter 16)

Chapter 16 continues the theory:

- Introduces **desirable properties of decompositions**:
 - **Nonadditive (lossless) join property (NJB)** — ensures no data is lost when decomposing.
 - **Dependency preservation** — ensures all FDs remain enforceable.
- Presents **algorithms** to test these properties and to design relations in desired normal forms.

- Discusses **minimal covers** of FDs.
- Adds **other dependency types** for advanced schema evaluation.

If Chapter 16 isn't studied in depth, at least understand **Property NJB** (Nonadditive Join Property).

◆ Summary of Key Concepts Introduced

Concept	Meaning
Functional Dependency (FD)	A formal relationship expressing that one attribute (or group) uniquely determines another.
Normalization	Systematic process of decomposing relations into smaller, well-structured relations that avoid redundancy and anomalies.
Normal Forms (1NF-5NF)	Levels of database design quality; higher forms remove more redundancy.
Information Preservation	Ensuring all data meaning from conceptual design remains in the relational design.
Lossless Decomposition	Ensures no information is lost when a relation is broken into smaller ones.

Would you like me to next explain **Section 15.1 (Informal Design Guidelines for Relation Schemas)** — which discusses *good vs. bad schema design principles* before getting into functional dependencies?

08/11/2025, 09:15:26



15.1 Informal Design Guidelines for Relation Schemas

Before discussing the formal theory of relational database design, we discuss four informal guidelines that may be used as measures to determine the quality of relation schema design:

- Making sure that the semantics of the attributes is clear in the schema
- Reducing the redundant information in tuples
- Reducing the NULL values in tuples
- Disallowing the possibility of generating spurious tuples

These measures are not always independent of one another, as we will see.

15.1.1 Imparting Clear Semantics to Attributes in Relations

Whenever we group attributes to form a relation schema, we assume that attributes belonging to one relation have certain real-world meaning and a proper interpretation associated with them. The semantics of a relation refers to its meaning resulting from the interpretation of attribute values in a tuple. In Chapter 3 we discussed how a relation can be interpreted as a set of facts. If the conceptual design described in Chapters 7 and 8 is done carefully and the mapping procedure in Chapter 9 is followed systematically, the relational schema design should have a clear meaning.

504

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

In general, the easier it is to explain the semantics of the relation, the better the relation schema design will be. To illustrate this, consider Figure 15.1, a simplified version of the COMPANY relational database schema in

Figure 3.5, and Figure 15.2, which presents an example of populated relation states of this schema. The meaning of the EMPLOYEE relation schema is quite simple: Each tuple represents an employee, with values for the employee's name (Ename), Social Security number (Ssn), birth date (Bdate), and address (Address), and the number of the department that the employee works for (Dnumber). The Dnumber attribute is a foreign key that represents an implicit relationship between EMPLOYEE and DEPARTMENT. The semantics of the DEPARTMENT and PROJECT schemas are also straightforward: Each DEPARTMENT tuple represents a department entity, and each PROJECT tuple represents a project entity. The attribute Dmgr_ssn of DEPARTMENT relates a department to the employee who is its manager, while Dnum of PROJECT relates a project to its controlling department; both are foreign key attributes. The ease with which the meaning of a relation's attributes can be explained is an informal measure of how well the relation is designed.

Figure 15.1

A simplified COMPANY relational database schema.

EMPLOYEE

Ename

Ssn

P.K.

DEPARTMENT

Dname

DEPT_LOCATIONS

F.K.

Dnumber Dlocation

P.K.

PROJECT

Pname Pnumber

P.K.
WORKS_ON
F.K.
Ssn
F.K.
Pnumber Hours
F.K.
Bdate
Dnumber
Address Dnumber
F.K.
Dmgr_ssn
P.K.
Plocation
F.K.
Dnum
P.K.

15.1 Informal Design Guidelines for Relation Schemas 505

Ename
EMPLOYEE
Smith, John B.
Wong, Franklin T.
Zelaya, Alicia J.
Wallace, Jennifer S.
Narayan, Ramesh K.
English, Joyce A.
Jabbar, Ahmad V.
Borg, James E.
999887777
123456789
333445555
453453453
987654321
666884444
987987987
888665555
666884444
123456789
123456789
333445555

453453453
453453453
333445555
333445555
333445555
999887777
987987987
999887777
987987987
987654321
987654321
888665555
3
1
2
2
1
2
3
10
20
10
30
10
30
30
20
20
40.0
32.5
7.5
10.0
20.0
20.0
10.0
10.0
10.0
35.0
30.0
10.0
5.0

20.0
15.0
Null
1937-11-10
1968-07-19
1965-01-09
1955-12-08
1972-07-31
1969-03-29
1941-06-20
1962-09-15
Bdate
3321 Castle, Spring, TX
731 Fondren, Houston, TX 5
638 Voss, Houston, TX
5631 Rice, Houston, TX
980 Dallas, Houston, TX
450 Stone, Houston, TX
291Berry, Bellaire, TX
975 Fire Oak, Humble, TX
Address
4
5
5
4
1
4
5
Dnumber
Dname
DEPARTMENT
Research
Administration
Headquarters 888665555
333445555
987654321
Dnumber
5
1
4
DEPT_LOCATIONS

1
4
5
Dnumber
Houston
Dlocation
Bellaire
Stafford
Houston
Sugarland
5
5
PROJECT
ProductX
ProductY
ProductZ
Pname
1
Pnumber Plocation Dnum
3
2
20
10
Reorganization
30
5
5
5
1
4
4
Bellaire
Houston
Sugarland
Houston
Stafford
Stafford Newbenefits
Computerization
WORKS_ON
Pnumber Hours
Ssn

Dmgr_ssn

Ssn

Figure 15.2

Sample database state for the relational database schema in Figure 15.1.

506

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

The semantics of the other two relation schemas in Figure 15.1 are slightly more complex. Each tuple in DEPT_LOCATIONS gives a department number (Dnumber) and one of the locations of the department (Dlocation). Each tuple in WORKS_ON gives an employee Social Security number (Ssn), the project number of one of the projects that the employee works on (Pnumber), and the number of hours per week that the employee works on that project (Hours). However, both schemas have a well-defined and unambiguous interpretation. The schema DEPT_LOCATIONS represents a multivalued attribute of DEPARTMENT, whereas WORKS_ON represents an M:N relationship between EMPLOYEE and PROJECT. Hence, all the relation schemas in Figure 15.1 may be considered as easy to explain and therefore good from the standpoint of having clear semantics. We can thus formulate the following informal design guideline.

Guideline 1

Design a relation schema so that it is easy to explain its meaning. Do not combine attributes from multiple entity types and relationship types into a single relation.

Intuitively, if a relation schema corresponds to one entity type or one relationship type, it is straightforward to interpret and to explain its meaning. Otherwise, if the relation corresponds to a mixture of multiple entities

and relationships, semantic ambiguities will result and the relation cannot be easily explained.

Examples of Violating Guideline 1. The relation schemas in Figures 15.3(a) and 15.3(b) also have clear semantics. (The reader should ignore the lines under the relations for now; they are used to illustrate functional dependency notation, discussed in Section 15.2.) A tuple in the EMP_DEPT relation schema in Figure 15.3(a) represents a single employee but includes additional information—namely, the name (Dname) of the department for which the employee works and the Social Security number (Dmgr_ssn) of the department manager. For the EMP_PROJ relation in Figure 15.3(b), each tuple relates an employee to a project but also includes

Figure 15.3

Two relation schemas suffering from update anomalies. (a)

EMP_DEPT and (b) EMP_PROJ.

(a)

EMP_DEPT

Ename

(b)

EMP_PROJ

Ssn

FD1

FD2

FD3

Ssn

Bdate Address Dnumber Dname

Pnumber Hours Ename Pname Plocation

Dmgr_ssn

15.1 Informal Design Guidelines for Relation Schemas

507

the employee name (Ename), project name

(Pname), and project location (Plocation).

Although there is nothing wrong logically with these two relations, they violate

Guideline 1 by mixing attributes from distinct real-world entities: EMP_DEPT mixes

attributes of employees and departments, and

EMP_PROJ mixes attributes of

employees and projects and the WORKS_ON

relationship. Hence, they fare poorly

against the above measure of design quality. They

may be used as views, but they

cause problems when used as base relations, as we discuss in the following section.

08/11/2025, 09:25:08



Excellent — this section (**15.1.1 Imparting Clear Semantics to Attributes in Relations**) introduces the **first informal guideline** for designing good relational schemas.

Let's unpack it in a clear, structured way 📌

◆ 15.1 Informal Design Guidelines for Relation Schemas

Before formalizing design theory with **functional dependencies (FDs)** and **normal forms**, we use **four informal guidelines** to evaluate the quality of a relation schema:

1. **Clear semantics** — Each relation should have a well-understood meaning.
2. **Minimal redundancy** — Avoid repeating the same information.
3. **Minimal nulls** — Avoid unnecessary null values.
4. **No spurious tuples** — Avoid designs that can produce incorrect data when relations are joined.

These guidelines are not completely independent; improving one can affect the others.

◆ 15.1.1 Imparting Clear Semantics to Attributes in Relations

🧩 Meaning

When designing a relation schema, the **attributes grouped together** should correspond to **one real-world entity or relationship**.

That is, **all attributes in a relation should describe the same concept**.

- The **semantics** of a relation = its *meaning* derived from how the attribute values in each tuple relate to real-world facts.

If your conceptual (ER/EER) design was done carefully and correctly mapped to relations, the semantics of each relation will naturally be clear.

🧠 Example: COMPANY Database (Figure 15.1 and 15.2)

Let's understand this with the **simplified COMPANY schema**:

Relation	Represents	Attributes
EMPLOYEE	Each employee	Ename, Ssn, Bdate, Address, Dnumber
DEPARTMENT	Each department	Dname, Dnumber, Dmgr_ssn
PROJECT	Each project	Pname, Pnumber, Plocation, Dnum
DEPT_LOCATIONS	Department's locations (multivalued attribute of department)	Dnumber, Dlocation
WORKS_ON	M:N relationship between employees and projects	Ssn, Pnumber, Hours

Each of these relations:

- Has a **clear and single meaning**.
- Can be **easily explained** in one sentence.
- Represents **one entity type or one relationship type**.

For example:

“Each tuple in EMPLOYEE represents one employee and their basic details.”

“Each tuple in WORKS_ON represents one employee working on one project for a given number of hours per week.”

Thus, these schemas **satisfy Guideline 1**.

Guideline 1 (Formal Statement)

Design a relation schema so that it is easy to explain its meaning.

Do not combine attributes from multiple entity types or relationship types into a single relation.

Intuition:

- If a relation corresponds to **one entity** (like EMPLOYEE) or **one relationship** (like WORKS_ON), its meaning is clear.
- If it combines attributes from **multiple entities or relationships**, it becomes ambiguous and error-prone.

Violating Guideline 1

Two examples of *bad design* are shown in **Figure 15.3(a) and 15.3(b)**:

(a) EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	-----	-------	---------	---------	-------	----------

This relation mixes:

- Employee information (Ename, Ssn, Bdate, Address)
- Department information (Dname, Dmgr_ssn)

✅ It is *logically valid* (no contradiction).

❌ But it mixes two separate entities (EMPLOYEE + DEPARTMENT).

Hence, it's **semantically unclear** — not easy to explain in one sentence.

(b) EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
-----	---------	-------	-------	-------	-----------

This relation mixes:

- Employee info (Ssn, Ename)
- Project info (Pnumber, Pname, Plocation)
- Relationship info (Hours — part of WORKS_ON)

✅ Again, logically valid.

❌ But semantically confusing — it combines three concepts: employee, project, and works_on.

Such relations are **okay as views** (derived tables for convenience), but **bad as base relations** (physically stored tables), because:

- They cause **update anomalies** (insertion, deletion, modification issues).
- They violate **Guideline 1**.

✅ Summary

Aspect	Good Design	Bad Design
Semantics	One clear meaning per relation	Mixed meanings
Entities per relation	One (entity or relationship)	Multiple
Interpretability	Easy to explain	Hard to explain
Example (good)	EMPLOYEE(Ename, Ssn, Bdate, Address, Dnumber)	
Example (bad)	EMP_DEPT(Ename, Ssn, Bdate, Address, Dnumber, Dname, Dmgr_ssn)	

Would you like me to explain the **next section (15.1.2 - Reducing Redundant Information in Tuples)**, which discusses how combining unrelated attributes leads to *update anomalies* (insertion, deletion, modification)?

08/11/2025, 09:25:10



15.1.2 Redundant Information in Tuples and Update Anomalies

One goal of schema design is to minimize the storage space used by the base relations (and hence the corresponding files). Grouping attributes into relation schemas has a significant effect on storage space. For example, compare the space used by the two base relations EMPLOYEE and DEPARTMENT in Figure 15.2 with that for an EMP_DEPT base relation in Figure 15.4, which is the result of applying the NATURAL JOIN operation to EMPLOYEE and DEPARTMENT. In EMP_DEPT, the attribute values pertaining to a particular department (Dnumber, Dname, Dmgr_ssn) are repeated for every employee who works for that department. In contrast, each department's information appears only once in the DEPARTMENT relation in Figure 15.2. Only the department number (Dnumber) is repeated in the EMPLOYEE relation for each employee who works in that department as a foreign key. Similar comments apply to the EMP_PROJ relation (see Figure 15.4), which augments the WORKS_ON relation with additional attributes from EMPLOYEE and PROJECT.

Storing natural joins of base relations leads to an additional problem referred to as update anomalies. These can be classified into insertion anomalies, deletion anomalies, and modification anomalies.²

Insertion Anomalies. Insertion anomalies can be differentiated into two types, illustrated by the following examples based on the EMP_DEPT relation:

- To insert a new employee tuple into EMP_DEPT, we must include either the

attribute values for the department that the employee works for, or NULLs (if the employee does not work for a department as yet). For example, to insert a new tuple for an employee who works in department number 5, we must enter all the attribute values of department 5 correctly so that they are consistent with the corresponding values for department 5 in other tuples in EMP_DEPT. In the design of Figure 15.2, we do not have to worry about this consistency problem because we enter only the department number in the employee tuple; all other attribute values of department 5 are recorded only once in the database, as a single tuple in the DEPARTMENT relation.

■ It is difficult to insert a new department that has no employees as yet in the EMP_DEPT relation. The only way to do this is to place NULL values in the

2These anomalies were identified by Codd (1972a) to justify the need for normalization of relations, as we shall discuss in Section 15.3.

508 Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

Ename

EMP_DEPT

Smith, John B.

Wong, Franklin T.

Zelaya, Alicia J.

Wallace, Jennifer S.

Narayan, Ramesh K.

English, Joyce A.

Jabbar, Ahmad V.

Borg, James E.

999887777

123456789

333445555

453453453

987654321
666884444
987987987
888665555 1937-11-10
Ssn
1968-07-19
1965-01-09
1955-12-08
1972-07-31
1969-03-29
1941-06-20
1962-09-15
Bdate
3321 Castle, Spring, TX
731 Fondren, Houston, TX 5
638 Voss, Houston, TX
5631 Rice, Houston, TX
980 Dallas, Houston, TX
450 Stone, Houston, TX
291 Berry, Bellaire, TX
975 FireOak, Humble, TX
Address
4
5
5
4
1
4
5
Administration
Research
Research
Research
Administration
Headquarters
Administration
Research
987654321
333445555
333445555
333445555

987654321
888665555
987654321
333445555
Dnumber Dname Dmgr_ssn
Ssn
EMP_PROJ
123456789
123456789
666884444
453453453
453453453
333445555
333445555
333445555
333445555
999887777
999887777
987987987
987987987
987654321
987654321
888665555
3
1
2
2
1
2
30
30
30
10
10
3
10
20
20
20
Pnumber
40.0

32.5
7.5
10.0
10.0
10.0
10.0
20.0
20.0
30.0
5.0
10.0
35.0
20.0
15.0
Null
Hours
Narayan, Ramesh K.
Smith, John B.
Smith, John B.
Wong, Franklin T.
Wong, Franklin T.
Wong, Franklin T.
Wong, Franklin T.
English, Joyce A.
English, Joyce A.
Zelaya, Alicia J.
Jabbar, Ahmad V.
Zelaya, Alicia J.
Jabbar, Ahmad V.
Wallace, Jennifer S.
Wallace, Jennifer S.
Borg, James E.
Ename
ProductZ
ProductX
ProductY
ProductY
ProductZ
Reorganization
ProductX
ProductY

Newbenefits
 Newbenefits
 Computerization
 Computerization
 Newbenefits
 Reorganization
 Reorganization
 Houston
 Bellaire
 Sugarland
 Sugarland
 Houston
 Stafford
 Houston
 Bellaire
 Sugarland
 Stafford
 Stafford
 Stafford
 Stafford
 Stafford
 Houston
 Houston
 Pname Plocation
 Computerization
 Redundancy Redundancy
 Redundancy

Figure 15.4

Sample states for EMP_DEPT and EMP_PROJ resulting from applying NATURAL JOIN to the relations in Figure 15.2. These may be stored as base relations for performance reasons.

attributes for employee. This violates the entity integrity for EMP_DEPT

because Ssn is its primary key. Moreover, when the first employee is assigned to that department, we do not need this tuple with NULL values any more.

This problem does not occur in the design of Figure 15.2 because a department is entered in the DEPARTMENT relation whether or

not any employees

work for it, and whenever an employee is assigned to that department, a corresponding tuple is inserted in EMPLOYEE.

15.1 Informal Design Guidelines for Relation Schemas

509

Deletion Anomalies. The problem of deletion anomalies is related to the second insertion anomaly situation just discussed. If we delete from EMP_DEPT an

employee tuple that happens to represent the last employee working for a particular department, the information concerning that department is lost from the database.

This problem does not occur in the database of Figure 15.2 because DEPARTMENT tuples are stored separately.

Modification Anomalies. In EMP_DEPT, if we change the value of one of the

attributes of a particular department—say, the manager of department 5—we must

update the tuples of all employees who work in that department; otherwise, the

database will become inconsistent. If we fail to update some tuples, the same department will be shown to have two different values for manager in different employee tuples, which would be wrong.³

It is easy to see that these three anomalies are undesirable and cause difficulties to

maintain consistency of data as well as require unnecessary updates that can be

avoided; hence, we can state the next guideline as follows.

Guideline 2

Design the base relation schemas so that no insertion, deletion, or modification

anomalies are present in the relations. If any anomalies are present,⁴ note them

clearly and make sure that the programs that update the database will operate

correctly.

The second guideline is consistent with and, in a way, a restatement of the first

guideline. We can also see the need for a more formal approach to evaluating

whether a design meets these guidelines. Sections 15.2 through 15.4 provide these

needed formal concepts. It is important to note that these guidelines may some

times have to be violated in order to improve the performance of certain queries. If

EMP_DEPT is used as a stored relation (known otherwise as a materialized view) in

addition to the base relations of EMPLOYEE and DEPARTMENT, the anomalies in

EMP_DEPT must be noted and accounted for (for example, by using triggers or

stored procedures that would make automatic updates). This way, whenever the

base relation is updated, we do not end up with inconsistencies. In general, it is

advisable to use anomaly-free base relations and to specify views that include the

joins for placing together the attributes frequently referenced in important queries.

08/11/2025, 09:28:45



Excellent — this section (**15.1.2 Redundant Information in Tuples and Update Anomalies**) introduces the **second informal design guideline** for relational schema design.

Let's break it down carefully and clearly 📌

◆ 15.1.2 Redundant Information in Tuples and Update Anomalies

🧩 The Main Idea

A **good schema design** should **minimize redundancy** — i.e., avoid **storing the same information repeatedly** in multiple tuples.

Why?

Because redundancy:

- **Wastes storage space**
- **Causes data inconsistency**
- **Leads to update anomalies**

🧠 Example: Comparing EMPLOYEE-DEPARTMENT vs. EMP_DEPT

In Figure 15.2:

- We have two **separate relations**:
 - `EMPLOYEE(Ename, Ssn, Bdate, Address, Dnumber)`
 - `DEPARTMENT(Dname, Dnumber, Dmgr_ssn)`
- The connection between them is by the **foreign key Dnumber**.

In Figure 15.4:

- We store the **NATURAL JOIN** of these two relations as **EMP_DEPT**:

```
EMP_DEPT(Ename, Ssn, Bdate, Address, Dnumber, Dname, Dmgr_ssn)
```

🧱 Redundancy Example

In `EMP_DEPT`, for every employee in a department, the **department information** (`Dnumber, Dname, Dmgr_ssn`) is repeated.

For example:

Ename	Dnumber	Dname	Dmgr_ssn
Smith, John B.	5	Research	333445555
Zelaya, Alicia J.	5	Research	333445555
Jabbar, Ahmad V.	5	Research	333445555

Here, “Research” and its manager’s SSN appear **three times**.
That is **redundant information**.

By contrast, in the **normalized design** (Figure 15.2):

- Department data appears only **once** (in DEPARTMENT),
- Employees only reference it through **Dnumber**.

⚠ **The Problem: Update Anomalies**

When redundant data is stored in one table, it causes **insertion, deletion, and modification anomalies**.

◆ **1. Insertion Anomalies**

When adding new data, you are forced to add **unnecessary or inconsistent** information.

Example 1 — New Employee

- You want to insert a new employee into `EMP_DEPT`.
- You must also enter **the department details** (`Dname`, `Dmgr_ssn`).
- If you mistype the department name or manager’s SSN, you create **inconsistent data**.

Example 2 — New Department

- You want to add a **new department** that currently has **no employees**.
- But `EMP_DEPT`’s **primary key is Ssn** — you can’t insert a department alone.
- You’d have to insert NULLs for employee data, which **violates entity integrity** (primary key cannot be NULL).

✅ In the normalized version (EMPLOYEE + DEPARTMENT), both insertions are easy and consistent:

- Add a department directly into DEPARTMENT.
- Add an employee into EMPLOYEE, referencing the department by Dnumber.

◆ 2. Deletion Anomalies

When deleting one fact, you **unintentionally delete other important data**.

Example:

If we delete the **last employee** who works in department 5 from `EMP_DEPT`,
→ we also lose **all information about department 5** (since no tuple now contains its data).

✅ In the normalized design, deleting an employee does not delete the department, since they're stored separately.

◆ 3. Modification (Update) Anomalies

When you change a value in one place, you must change it in **many tuples**, or else your database becomes **inconsistent**.

Example:

If the **manager of department 5** changes,
→ you must update that value (`Dmgr_ssn`) in **every EMP_DEPT tuple** where Dnumber = 5.

If you forget one row, the database shows **two different managers** for the same department.

✅ In the normalized design, the manager's SSN is stored only once — in DEPARTMENT — so a single update suffices.

🚀 Summary of Anomalies

Type of Anomaly	What Happens	Why It's Bad
Insertion	Can't insert new departments or must repeat dept. info for new employee	Violates integrity or causes inconsistency
Deletion	Deleting last employee removes department info	Unintended data loss
Modification	Must change repeated data in many rows	Inconsistent data possible

✅ Guideline 2 (Formal Statement)

Design the base relation schemas so that no insertion, deletion, or modification anomalies are present.

*If any anomalies are present, **document them clearly** and ensure **programs or triggers handle them correctly**.*

🧠 Relation to Guideline 1

- **Guideline 1:** Don't mix unrelated entities → avoids semantic confusion.
- **Guideline 2:** Don't store redundant information → avoids update anomalies.

They are **closely related** — violating the first usually leads to violating the second.

⚙️ Practical Note

Sometimes, designers **intentionally denormalize** (violate the guidelines) to **improve query performance**.

For example, storing `EMP_DEPT` as a **materialized view** for faster joins.

In that case:

- The anomalies must be handled automatically — e.g., using **triggers** or **stored procedures**.
- Base relations should still remain **anomaly-free**.

📄 In Short

Concept	Description
Redundancy	Repeated storage of the same facts
Effect	Causes inconsistency and extra storage
Types of anomalies	Insertion, Deletion, Modification
Solution	Normalize relations (split logically into entities and relationships)
Guideline 2	Avoid designs with update anomalies

Would you like me to explain **Section 15.1.3 - Reducing the NULL Values in Tuples** next? It continues this topic by showing how poor design can cause excessive nulls and waste space.

08/11/2025, 09:28:48



NULL Values in Tuples

In some schema designs we may group many attributes together into a “fat” relation. If many of the attributes do not apply to all tuples in the relation, we end up with many NULLs in those tuples. This can waste space at the storage level and may

3This is not as serious as the other problems, because all tuples can be updated by a single SQL query.

4Other application considerations may dictate and make certain anomalies unavoidable. For example, the EMP_DEPT relation may correspond to a query or a report that is frequently required.

510

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

also lead to problems with understanding the meaning of the attributes and with

specifying JOIN operations at the logical level.⁵

Another problem with NULLs is how

to account for them when aggregate operations such as COUNT or SUM are applied.

SELECT and JOIN operations involve comparisons; if NULL values are present, the

results may become unpredictable.⁶ Moreover, NULLs can have multiple interpretations, such as the following:

- The attribute does not apply to this tuple. For example, Visa_status may not apply to U.S. students.

- The attribute value for this tuple is unknown. For example, the Date_of_birth may be unknown for an employee.

- The value is known but absent; that is, it has not been recorded yet. For example, the Home_Phone_Number for an employee may exist, but may not be available and recorded yet.

Having the same representation for all NULLs

compromises the different meanings they may have. Therefore, we may state another guideline.

Guideline 3

As far as possible, avoid placing attributes in a base relation whose values may frequently be NULL. If NULLs are unavoidable, make sure that they apply in exceptional cases only and do not apply to a majority of tuples in the relation.

Using space efficiently and avoiding joins with NULL values are the two overriding criteria that determine whether to include the columns that may have NULLs in a relation or to have a separate relation for those columns (with the appropriate key columns). For example, if only 15 percent of employees have individual offices, there is little justification for including an attribute `Office_number` in the `EMPLOYEE` relation; rather, a relation `EMP_OFFICES(Essn, Office_number)` can be created to include tuples for only the employees with individual offices.

15.1.4 Generation of Spurious Tuples

Consider the two relation schemas `EMP_LOCS` and `EMP_PROJ1` in Figure 15.5(a), which can be used instead of the single `EMP_PROJ` relation in Figure 15.3(b). A tuple in `EMP_LOCS` means that the employee whose name is `Ename` works on some project whose location is `Plocation`. A tuple in `EMP_PROJ1` refers to the fact that the employee whose Social Security number is `Ssn` works `Hours per week` on the project whose name, number, and location are `Pname`, `Pnumber`, and `Plocation`. Figure

15.5(b) shows relation states of `EMP_LOCS` and `EMP_PROJ1` corresponding to the

5This is because inner and outer joins produce different results when NULLs are involved in joins. The

users must thus be aware of the different meanings of the various types of joins. Although this is reasonable for sophisticated users, it may be difficult for others.

In Section 5.5.1 we presented comparisons involving NULL values where the outcome (in three-valued logic) are TRUE, FALSE, and UNKNOWN.

15.1 Informal Design Guidelines for Relation Schemas

511

(a)

EMP_LOCS

Ename

P.K.

EMP_PROJ1

Ssn Pnumber Hours Pname Plocation

P.K.

(b)

EMP_LOCS

Ename

Smith, John B.

Smith, John B.

Narayan, Ramesh K.

English, Joyce A.

English, Joyce A.

Wong, Franklin T.

Wong, Franklin T.

Wong, Franklin T.

Zelaya, Alicia J.

Jabbar, Ahmad V.

Wallace, Jennifer S.

Wallace, Jennifer S.

Borg, James E.

Figure 15.5

Plocation

Particularly poor design for the EMP_PROJ relation in

Figure 15.3(b). (a) The two relation schemas

EMP_LOCS

and EMP_PROJ1. (b) The result of projecting the extension of EMP_PROJ from Figure 15.4 onto the relations

EMP_LOCS and EMP_PROJ1.

EMP_PROJ1

Plocation
Bellaire
Sugarland
Houston
Bellaire
Sugarland
Sugarland
Houston
Stafford
Stafford
Stafford
Stafford
Houston
Houston
Ssn
123456789
123456789
666884444
453453453
453453453
333445555
333445555
333445555
333445555
999887777
999887777
987987987
987987987
987654321
987654321
Pnumber
1
2
3
1
2
2
3
10
20
30

10
10
Hours
32.5
7.5
40.0
20.0
20.0
10.0
10.0
10.0
10.0
30.0
10.0
35.0
Pname
ProductX
ProductY
ProductZ
ProductX
ProductY
ProductY
ProductZ
Computerization
Reorganization
Newbenefits
Computerization
Computerization
Plocation
Bellaire
Sugarland
Houston
Bellaire
Sugarland
Sugarland
Houston
Stafford
Houston
Stafford
Stafford
Stafford

5.0
 888665555
 30
 30
 20
 20
 20.0
 15.0
 NULL
 Newbenefits
 Newbenefits
 Reorganization
 Reorganization
 Stafford
 Stafford
 Houston
 Houston

EMP_PROJ relation in Figure 15.4, which are obtained by applying the appropriate PROJECT (π) operations to EMP_PROJ (ignore the dashed lines in Figure 15.5(b) for now).

Suppose that we used EMP_PROJ1 and EMP_LOCS as the base relations instead of EMP_PROJ. This produces a particularly bad schema design because we cannot recover the information that was originally in EMP_PROJ from EMP_PROJ1 and EMP_LOCS. If we attempt a NATURAL JOIN operation on EMP_PROJ1 and EMP_LOCS, the result produces many more tuples than the original set of tuples in EMP_PROJ. In Figure 15.6, the result of applying the join to only the tuples above the dashed lines in Figure 15.5(b) is shown (to reduce the size of the resulting relation). Additional tuples that were not in EMP_PROJ are called spurious tuples

512 Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases
 Ssn

123456789
123456789
123456789
123456789
123456789
666884444
666884444
453453453
453453453
453453453
453453453
453453453
333445555
333445555
333445555
333445555
2
1
1
3
2
2
2
2
2
2
2
2
3
1
1
2
3
Pnumber
7.5
32.5
32.5
40.0
40.0
20.0
20.0
7.5

7.5
20.0
10.0
20.0
20.0
10.0
10.0
10.0
Hours
ProductY
ProductX
ProductX
ProductZ
ProductZ
ProductX
ProductX
ProductY
ProductY
ProductY
ProductY
ProductY
ProductY
ProductY
ProductY
ProductY
ProductZ
Pname
Sugarland
Bellaire
Bellaire
Houston
Houston
Bellaire
Bellaire
Sugarland
Sugarland
Sugarland
Sugarland
Sugarland
Sugarland
Sugarland
Sugarland
Sugarland

Houston
333445555 3 10.0 ProductZ Houston
333445555 10 10.0 Computerization Stafford
333445555 20 10.0 Reorganization Houston
333445555 20

*
*
*
*
*
*
*
*
*
*
*
*

10.0 Reorganization Houston

Smith, John B.
Smith, John B.
English, Joyce A.
Narayan, Ramesh K.
Wong, Franklin T.
Smith, John B.
English, Joyce A.
English, Joyce A.
Wong, Franklin T.
Smith, John B.
Smith, John B.
English, Joyce A.
Wong, Franklin T.
English, Joyce A.
Wong, Franklin T.
Narayan, Ramesh K.
Wong, Franklin T.
Wong, Franklin T.
Narayan, Ramesh K.
Wong, Franklin T.
Plocation Ename

* * *

Figure 15.6

Result of applying NATURAL JOIN to the tuples above

the dashed lines
in EMP_PROJ1 and EMP_LOCS of Figure 15.5. Generated
spurious
tuples are marked by asterisks.
because they represent spurious information that is
not valid. The spurious tuples
are marked by asterisks (*) in Figure 15.6.

Decomposing EMP_PROJ into
EMP_LOCS and EMP_PROJ1 is undesirable because
when we JOIN them back using NATURAL JOIN, we do
not get the correct original
information. This is because in this case Plocation is the
attribute that relates
EMP_LOCS and EMP_PROJ1, and Plocation is neither a
primary key nor a foreign
key in either EMP_LOCS or EMP_PROJ1. We can now
informally state another
design guideline.

Guideline 4
Design relation schemas so that they can be joined
with equality conditions on
attributes that are appropriately related (primary key,
foreign key) pairs in a way
that guarantees that no spurious tuples are
generated. Avoid relations that contain

15.2 Functional Dependencies

513

matching attributes that are not (foreign key, primary
key) combinations because
joining on such attributes may produce spurious
tuples.

This informal guideline obviously needs to be stated
more formally. In Section 16.2
we discuss a formal condition called the nonadditive
(or lossless) join property that
guarantees that certain joins do not produce spurious
tuples.

08/11/2025, 09:35:13



Excellent — this passage continues the *informal design guidelines* for relational schemas, specifically **Guideline 3** and **Guideline 4**. Let's break it down clearly and concisely so you understand what's going on.

Section Summary: *NULL Values in Tuples* (Guideline 3)

The Problem

When too many attributes are combined into one “fat” relation (wide table), some tuples may have **many NULL values** because:

- Certain attributes don't apply to every entity.
- Some values are **unknown**.
- Some values **exist but haven't been recorded yet**.

Example Interpretations of NULL:

Case	Meaning	Example
Attribute not applicable	The attribute doesn't make sense for that tuple	<code>Visa_status</code> doesn't apply to U.S. students
Value unknown	Data is missing	Employee's <code>Date_of_birth</code> unknown
Value exists but not recorded yet	Data will be filled later	Employee's <code>Home_Phone_Number</code> not entered yet

Having **one common NULL** for all three meanings can confuse query results and logic.

Problems Caused by NULLs

1. **Wasted storage space** (many empty fields).
2. **Ambiguity** in interpreting what NULL means.
3. **Issues in operations** like `JOIN`, `SUM`, or `COUNT` — results may become unpredictable.

4. **Difficulties for users** when designing queries involving NULLs (INNER vs OUTER joins behave differently).

✅ **Guideline 3 (Avoid NULLs)**

As far as possible, avoid placing attributes in a base relation whose values may frequently be NULL.

If NULLs are unavoidable:

- They should appear only **in exceptional cases**.
- Consider **creating a separate relation** for those attributes.

🧩 **Example**

Instead of:

```
EMPLOYEE(Essn, Name, Office_number)
```

(where only 15% have offices → 85% NULLs)

Better design:

```
EMPLOYEE(Essn, Name)
EMP_OFFICES(Essn, Office_number)
```

This saves space and avoids unnecessary NULLs.

✂ **Section Summary: *Generation of Spurious Tuples* (Guideline 4)**

📖 **The Problem**

When we decompose a relation incorrectly, **joining them back** may produce **spurious (false) tuples** — i.e., extra rows that **did not exist in the original data**.

🧩 **Example Scenario**

They show a poor decomposition:

```
EMP_PROJ → EMP_LOCS(Ename, Plocation)
           EMP_PROJ1(Ssn, Pnumber, Hours, Pname, Plocation)
```


Here, `Plocation` is the **common attribute**.

But it's **not a key** (neither primary nor foreign).

When we `NATURAL JOIN` on `Plocation`, the result mixes data incorrectly — creating **spurious tuples** (rows with wrong combinations of employee, project, and location).

In Figure 15.6, tuples marked with `*` are those false combinations.

Why It Happens

Because the join condition (`Plocation`) does **not uniquely identify** matching records in both relations.

Guideline 4 (Avoid Spurious Tuples)

*Design relation schemas so that they can be joined using equality conditions on attributes that are **primary key-foreign key** pairs, in a way that **guarantees no spurious tuples** are generated.*

That means:

- Only join on **key attributes**.
- Avoid using attributes like `Plocation` that are not uniquely identifying.

Related Formal Concept

Later, this is made precise in normalization theory as the **Lossless (Nonadditive) Join Property** — it ensures decomposed relations, when joined, reconstruct the original relation **without data loss or spurious rows**.

Summary of Both Guidelines

Guideline	Problem Addressed	Key Idea
3	Too many NULL values	Avoid attributes that are NULL for many tuples; separate them if needed
4	Spurious tuples after JOIN	Decompose relations so that joins use primary-foreign key links only

Would you like me to explain **Guideline 1 and 2** also (the ones before these) in the same structured and summarized way, so you have all four together clearly?

08/11/2025, 09:35:17



15.1.5 Summary and Discussion of Design Guidelines

In Sections 15.1.1 through 15.1.4, we informally discussed situations that lead to problematic relation schemas and we proposed informal guidelines for a good relational design. The problems we pointed out, which can be detected without additional tools of analysis, are as follows:

- Anomalies that cause redundant work to be done during insertion into and modification of a relation, and that may cause accidental loss of information during a deletion from a relation
- Waste of storage space due to NULLs and the difficulty of performing selections, aggregation operations, and joins due to NULL values
- Generation of invalid and spurious data during joins on base relations with matched attributes that may not represent a proper (foreign key, primary key) relationship

In the rest of this chapter we present formal concepts and theory that may be used to define the goodness and badness of individual relation schemas more precisely.

First we discuss functional dependency as a tool for analysis. Then we specify the three normal forms and Boyce-Codd normal form (BCNF) for relation schemas.

The strategy for achieving a good design is to decompose a badly designed relation appropriately. We also briefly introduce additional normal forms that deal with additional dependencies. In Chapter 16, we discuss the properties of decomposition in detail, and provide algorithms that design relations bottom-up by using the functional dependencies as a starting point



Perfect — this section (**15.1.5 Summary and Discussion of Design Guidelines**) neatly wraps up the four informal guidelines you've been studying. Let's summarize and interpret it clearly:

15.1.5 Summary and Discussion of Design Guidelines

This section ties together everything from **Sections 15.1.1 to 15.1.4**, where the textbook discussed the *informal guidelines* for designing good relational schemas.

Main Problems in Poor Schema Design

The text lists **three major categories of problems** that can occur when the database schema is poorly designed:

1 Data Redundancy and Update Anomalies

- **What happens:**
 - The same information is stored **multiple times** in different tuples.
 - This causes **insertion, deletion, and modification anomalies**.
 - **Example:**
 - In a combined relation like `EMP_DEPT`, the department's name and manager ID are repeated for every employee in that department.
 - Updating the department's manager requires updating multiple rows — if one is missed, inconsistency occurs.
 - **Consequence:**

Wasted effort, inconsistency, and potential data loss.
-

2 NULL Values and Storage Waste

- **What happens:**

- When a table contains attributes that don't apply to all entities, many tuples have **NULLs**.
 - These cause:
 - **Wasted storage space**
 - **Difficulty in performing operations** like `JOIN`, `COUNT`, `SUM`
 - **Ambiguity** in interpreting what a NULL means (not applicable, unknown, or missing)
 - **Example:**
 - Storing `office_number` in `EMPLOYEE` when most employees don't have offices → many NULLs.
 - **Consequence:**
Poor space efficiency and confusing query behavior.
-

3 Spurious Tuples from Improper Decomposition

- **What happens:**
 - When a relation is **decomposed incorrectly** (e.g., joined on attributes that are not key-foreign key pairs), **spurious tuples** (invalid combinations) appear when re-joining them.
 - **Example:**
 - Joining `EMP_PROJ1` and `EMP_LOCS` on `Plocation` created **false tuples** that didn't exist in the original relation.
 - **Consequence:**
Incorrect or misleading data when queries are executed.
-

💡 Purpose of the Next Sections

The book now transitions from **informal guidelines** to **formal theory** — providing mathematical tools to detect and avoid these problems.

The tools and concepts introduced next:

- 1. **Functional Dependencies (FDs):**
To formally express relationships between attributes.
- 2. **Normal Forms (1NF, 2NF, 3NF, BCNF):**
To define “good” vs “bad” relation designs.
- 3. **Decomposition Strategies:**
To break down large or redundant relations into smaller, consistent ones.
- 4. **Later in Chapter 16:**
 - Discusses **lossless join** and **dependency preservation** — ensuring decomposed relations can still reconstruct the original information correctly.

 In Summary

Problem	Cause	Guideline	Solution
Update anomalies	Redundant data	1 & 2	Separate logically distinct data (normalize)
NULLs and wasted space	Irrelevant attributes in a relation	3	Avoid frequent NULLs; use separate tables
Spurious tuples	Bad decomposition (wrong join attributes)	4	Only decompose using key-foreign key relationships

Would you like me to give a **combined chart or concise summary of all four guidelines (1-4)** in a single table next, for quick revision before the functional dependency section?

08/11/2025, 09:43:22



15.2 Functional Dependencies

So far we have dealt with the informal measures of database design. We now introduce a formal tool for analysis of relational schemas that enables us to detect and describe some of the above-mentioned problems in precise terms. The single most important concept in relational schema design theory is that of a functional dependency. In this section we formally define the concept, and in Section 15.3 we see how it can be used to define normal forms for relation schemas.

15.2.1 Definition of Functional Dependency

A functional dependency is a constraint between two sets of attributes from the database. Suppose that our relational database schema has n attributes $A_1, A_2, \dots, A_n$; let us think of the whole database as being described by a single universal

514

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

relation schema $R = \{A_1, A_2, \dots, A_n\}$.⁷ We do not imply that we will actually store the database as a single universal table; we use this concept only in developing the formal theory of data dependencies.⁸

Definition. A functional dependency, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R specifies a constraint on the possible tuples that can form a relation state r of R . The constraint is that, for any two tuples t_1 and t_2 in r that have $t_1[X] = t_2[X]$, they must also have $t_1[Y] = t_2[Y]$.

This means that the values of the Y component of a tuple in r depend on, or are determined by, the values of the X component;

alternatively, the values of the X component of a tuple uniquely (or functionally) determine the values of the Y component. We also say that there is a functional dependency from X to Y, or that Y is functionally dependent on X. The abbreviation for functional dependency is FD or f.d. The set of attributes X is called the left-hand side of the FD, and Y is called the right-hand side.

Thus, X functionally determines Y in a relation schema R if, and only if, whenever two tuples of $r(R)$ agree on their X-value, they must necessarily agree on their Y value. Note the following:

■ If a constraint on R states that there cannot be more than one tuple with a given X-value in any relation instance $r(R)$ —that is, X is a candidate key of R—this implies that $X \rightarrow Y$ for any subset of attributes Y of R (because the key constraint implies that no two tuples in any legal state $r(R)$ will have the same value of X). If X is a candidate key of R, then $X \rightarrow R$.

■ If $X \rightarrow Y$ in R, this does not say whether or not $Y \rightarrow X$ in R.

A functional dependency is a property of the semantics or meaning of the attributes. The database designers will use their understanding of the semantics of the attributes of R—that is, how they relate to one another—to specify the functional dependencies that should hold on all relation states (extensions) r of R. Whenever the semantics of two sets of attributes in R indicate that a functional dependency should hold, we specify the dependency as a constraint. Relation extensions $r(R)$ that satisfy the functional dependency constraints are called legal relation states (or

legal extensions) of R. Hence, the main use of functional dependencies is to describe further a relation schema R by specifying constraints on its attributes that must hold at all times. Certain FDs can be specified without referring to a specific relation, but as a property of those attributes given their commonly understood meaning. For example, {State, Driver_license_number} $\rightarrow$ Ssn should hold for any adult in the United States and hence should hold whenever these attributes appear in a relation. It is also possible that certain functional dependencies may cease to hold. This concept of a universal relation is important when we discuss the algorithms for relational database design in Chapter 16.

This assumption implies that every attribute in the database should have a distinct name. In Chapter 3 we prefixed attribute names by relation names to achieve uniqueness whenever attributes in distinct relations had the same name.

15.2 Functional Dependencies

515

exist in the real world if the relationship changes. For example, the FD Zip_code $\rightarrow$ Area_code used to exist as a relationship between postal codes and telephone number codes in the United States, but with the proliferation of telephone area codes it is no longer true.

Consider the relation schema EMP_PROJ in Figure 15.3(b); from the semantics of the attributes and the relation, we know that the following functional dependencies should hold:

- a. Ssn $\rightarrow$ Ename
- b. Pnumber $\rightarrow$ {Pname, Plocation}
- c. {Ssn, Pnumber} $\rightarrow$ Hours

These functional dependencies specify that (a) the value of an employee's Social

Security number (Ssn) uniquely determines the employee name (Ename), (b) the value of a project's number (Pnumber) uniquely determines the project name (Pname) and location (Plocation), and (c) a combination of Ssn and Pnumber values uniquely determines the number of hours the employee currently works on the project per week (Hours). Alternatively, we say that Ename is functionally determined by (or functionally dependent on) Ssn, or given a value of Ssn, we know the value of Ename, and so on.

A functional dependency is a property of the relation schema R, not of a particular legal relation state r of R. Therefore, an FD cannot be inferred automatically from a given relation extension r but must be defined explicitly by someone who knows the semantics of the attributes of R. For example, Figure 15.7 shows a particular state of the TEACH relation schema. Although at first glance we may think that Text $\rightarrow$

Course, we cannot confirm this unless we know that it is true for all possible legal states of TEACH. It is, however, sufficient to demonstrate a single counterexample to disprove a functional dependency. For example, because 'Smith' teaches both 'Data Structures' and 'Data Management,' we can conclude that Teacher does not functionally determine Course.

Given a populated relation, one cannot determine which FDs hold and which do not unless the meaning of and the relationships among the attributes are known. All one can say is that a certain FD may exist if it holds in that particular extension. One cannot guarantee its existence until the meaning of the corresponding attributes is clearly understood. One can, however, emphatically

state that a certain FD does not

TEACH

Course

Figure 15.7

Teacher

Smith

Smith

Hall

Brown

Text

Data Structures

Data Management

Compilers

Data Structures

Bartram

Martin

Hoffman

Horowitz

A relation state of TEACH with a

possible functional dependency

TEXT $\rightarrow$ COURSE. However,

TEACHER $\rightarrow$ COURSE is ruled

out.

516

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

hold if there are tuples that show the violation of such an FD. See the illustrative

example relation in Figure 15.8. Here, the following

FDs may hold because the four

tuples in the current extension have no violation of these constraints: $B \rightarrow C$;

$C \rightarrow B$; $\{A, B\} \rightarrow C$; $\{A, B\} \rightarrow D$; and $\{C, D\} \rightarrow B$.

However, the following do not

hold because we already have violations of them in the given extension: $A \rightarrow B$

(tuples 1 and 2 violate this constraint); $B \rightarrow A$ (tuples 2 and 3 violate this con

straint); $D \rightarrow C$ (tuples 3 and 4 violate it).

Figure 15.3 introduces a diagrammatic notation for displaying FDs: Each FD is dis

played as a horizontal line. The left-hand-side attributes of the FD are connected by vertical lines to the line representing the FD, while the right-hand-side attributes are connected by the lines with arrows pointing toward the attributes.

We denote by F the set of functional dependencies that are specified on relation schema R . Typically, the schema designer specifies the functional dependencies that are semantically obvious; usually, however, numerous other functional dependencies hold in all legal relation instances among sets of attributes that can be derived from and satisfy the dependencies in F . Those other dependencies can be inferred or deduced from the FDs in F . We defer the details of inference rules and properties of functional dependencies to Chapter 16.

08/11/2025, 09:44:39



Here's a **clear, structured summary and explanation** of your excerpt on **Functional Dependencies (FDs)** from *Elmasri & Navathe – Fundamentals of Database Systems, Chapter 15.2* 📌

15.2 Functional Dependencies — Summary and Explanation

1. Purpose

Functional Dependencies (FDs) are the **formal tool** used to **analyze relational schemas**.

They help identify whether a schema is well-designed or suffers from redundancy, anomalies, or dependency issues.

2. Definition

A **Functional Dependency** between two sets of attributes X and Y in a relation schema R is written as:

$$X \rightarrow Y$$

It means:

For any two tuples t_1 and t_2 in relation $r(R)$,
if $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$.

✅ Interpretation:

The value of X uniquely determines the value of Y .

- X : *Determinant* (left-hand side)
- Y : *Dependent attributes* (right-hand side)

3. Key Points

Concept	Meaning
FD represents a constraint	It specifies how attributes are related and what dependencies must hold in all valid instances of a relation.
If X is a candidate key , then	$X \rightarrow Y$ for all $Y \subseteq R$.
FDs are semantic , not automatic	They depend on the <i>meaning</i> of attributes, not on observed data.

4. Example: EMP_PROJ Relation

Attributes:

{Ssn, Ename, Pnumber, Pname, Plocation, Hours}

Functional Dependencies:

1. $Ssn \rightarrow Ename$
→ Each employee's SSN uniquely identifies their name.
2. $Pnumber \rightarrow \{Pname, Plocation\}$
→ Each project number identifies one project and its location.
3. $\{Ssn, Pnumber\} \rightarrow Hours$
→ For each employee-project pair, the number of hours worked is fixed.

5. Important Observations

- A **functional dependency is a property of the schema**, not of a single instance (table).
It must hold for *all possible legal states* of the relation.
- **You cannot derive FDs from data alone.**
Data can *disprove* an FD (if violated), but cannot *prove* it universally true without semantic understanding.

6. Counterexamples and Violations

To **disprove** an FD, one counterexample (two tuples violating the rule) is enough.

Example:

If we have in relation **TEACH**:

Teacher	Course	Text
Smith	Data Structures	Horowitz
Smith	Data Management	Bartram

Then,

- **Teacher → Course** ❌ *does not hold* (since Smith teaches two courses).
- **Text → Course** ✅ *may hold* if each textbook is used for only one course.

7. Inferring New Dependencies

A schema may have many **implied** FDs that can be **derived** logically from the given set F .

These rules of inference (like *Armstrong's axioms*) are covered in Chapter 16.

8. Diagrammatic Notation

FDs can be represented graphically:

```
Ssn ----> Ename
Pnumber ----> Pname, Plocation
{Ssn, Pnumber} ----> Hours
```

This helps visualize dependencies and determine normalization needs.

9. Practical Insight

Functional Dependencies are crucial for:

- Detecting **update anomalies**.
 - Guiding **normalization** (1NF, 2NF, 3NF, BCNF).
 - Ensuring **data consistency** and **integrity**.
-

10. Example Summary (from Figure 15.8)

Given attributes A, B, C, D —

and sample tuples, we can check which FDs **hold** and which **violate**:

✅ Hold:

- $B \rightarrow C$
- $C \rightarrow B$
- $\{A, B\} \rightarrow C$
- $\{A, B\} \rightarrow D$
- $\{C, D\} \rightarrow B$

❌ Do not hold:

- $A \rightarrow B$
 - $B \rightarrow A$
 - $D \rightarrow C$
-

In Short

Concept	Description
Functional Dependency (FD)	Relationship where one set of attributes determines another
Purpose	To detect redundancy, anomalies, and poor design
Defined by	Database semantics, not by sample data
Used for	Normalization and schema refinement
Violation Test	One counterexample is enough to disprove an FD

Would you like me to show **how to test an FD on an example table step-by-step** (like verifying if $A \rightarrow B$ holds in a given relation)?

08/11/2025, 09:44:41



15.3.1 Normalization of Relations

The normalization process, as first proposed by Codd (1972a), takes a relation

schema through a series of tests to certify whether it satisfies a certain normal form.

The process, which proceeds in a top-down fashion by evaluating each relation

against the criteria for normal forms and decomposing relations as necessary, can

thus be considered as relational design by analysis.

Initially, Codd proposed three

normal forms, which he called first, second, and third normal form. A stronger def

inition of 3NF—called Boyce-Codd normal form (BCNF)—was proposed later by

Boyce and Codd. All these normal forms are based on a single analytical tool: the

functional dependencies among the attributes of a relation. Later, a fourth normal

form (4NF) and a fifth normal form (5NF) were

proposed, based on the concepts of

multivalued dependencies and join dependencies, respectively; these are briefly dis

cussed in Sections 15.6 and 15.7.

Normalization of data can be considered a process of analyzing the given relation

schemas based on their FDs and primary keys to achieve the desirable properties of

(1) minimizing redundancy and (2) minimizing the insertion, deletion, and update

anomalies discussed in Section 15.1.2. It can be

considered as a “filtering” or “purifi

cation” process to make the design have successively better quality. Unsatisfactory

relation schemas that do not meet certain conditions—the normal form tests—are

decomposed into smaller relation schemas that meet the tests and hence possess the

desirable properties. Thus, the normalization

procedure provides database designers with the following:

- A formal framework for analyzing relation schemas based on their keys and on the functional dependencies among their attributes
- A series of normal form tests that can be carried out on individual relation schemas so that the relational database can be normalized to any desired degree

Definition. The normal form of a relation refers to the highest normal form condition that it meets, and hence indicates the degree to which it has been normalized.

Normal forms, when considered in isolation from other factors, do not guarantee a good database design. It is generally not sufficient to check separately that each

518

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

relation schema in the database is, say, in BCNF or 3NF. Rather, the process of nor

malization through decomposition must also confirm the existence of additional

properties that the relational schemas, taken together, should possess. These would

include two properties:

- The nonadditive join or lossless join property, which guarantees that the

spurious tuple generation problem discussed in Section 15.1.4 does not

occur with respect to the relation schemas created after decomposition.

- The dependency preservation property, which ensures that each functional

dependency is represented in some individual relation resulting after

decomposition.

The nonadditive join property is extremely critical and

must be achieved at any cost, whereas the dependency preservation property, although desirable, is some times sacrificed, as we discuss in Section 16.1.2. We defer the presentation of the formal concepts and techniques that guarantee the above two properties to Chapter 16.

15.3.2 Practical Use of Normal Forms

Most practical design projects acquire existing designs of databases from previous designs, designs in legacy models, or from existing files. Normalization is carried out in practice so that the resulting designs are of high quality and meet the desirable properties stated previously. Although several higher normal forms have been defined, such as the 4NF and 5NF that we discuss in Sections 15.6 and 15.7, the practical utility of these normal forms becomes questionable when the constraints on which they are based are rare, and hard to understand or to detect by the database designers and users who must discover these constraints. Thus, database design as practiced in industry today pays particular attention to normalization only up to 3NF, BCNF, or at most 4NF.

Another point worth noting is that the database designers need not normalize to the highest possible normal form. Relations may be left in a lower normalization status, such as 2NF, for performance reasons, such as those discussed at the end of Section

15.1.2. Doing so incurs the corresponding penalties of dealing with the anomalies.

Definition. Denormalization is the process of storing the join of higher normal form relations as a base relation, which is in a lower normal form.

15.3.3 Definitions of Keys and Attributes Participating in Keys

Before proceeding further, let's look again at the definitions of keys of a relation schema from Chapter 3.

Definition. A superkey of a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes $S \subseteq R$ with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$. A key K is a superkey with the additional property that removal of any attribute from K will cause K not to be a superkey any more.

15.3 Normal Forms Based on Primary Keys

519

The difference between a key and a superkey is that a key has to be minimal; that is, if we have a key $K = \{A_1, A_2, \dots, A_k\}$ of R , then $K - \{A_i\}$ is not a key of R for any $A_i, 1 \leq i \leq k$. In Figure 15.1, $\{Ssn\}$ is a key for EMPLOYEE, whereas $\{Ssn\}$, $\{Ssn, Ename\}$, $\{Ssn, Ename, Bdate\}$, and any set of attributes that includes Ssn are all superkeys.

If a relation schema has more than one key, each is called a candidate key. One of the candidate keys is arbitrarily designated to be the primary key, and the others are called secondary keys. In a practical relational database, each relation schema must have a primary key. If no candidate key is known for a relation, the entire relation can be treated as a default superkey. In Figure 15.1, $\{Ssn\}$ is the only candidate key for EMPLOYEE, so it is also the primary key.

Definition. An attribute of relation schema R is called a prime attribute of R if it is a member of some candidate key of R . An attribute is called nonprime if it is not a prime attribute—that is, if it is not a member of any candidate key.

In Figure 15.1, both Ssn and $Pnumber$ are prime attributes of WORKS_ON, whereas

other attributes of WORKS_ON are nonprime.
We now present the first three normal forms: 1NF, 2NF, and 3NF. These were proposed by Codd (1972a) as a sequence to achieve the desirable state of 3NF relations by progressing through the intermediate states of 1NF and 2NF if needed. As we shall see, 2NF and 3NF attack different problems. However, for historical reasons, it is customary to follow them in that sequence; hence, by definition a 3NF relation already satisfies 2NF.

08/11/2025, 12:16:55



Here's a **clear, structured explanation and summary** of your excerpt on **Normalization of Relations (Section 15.3.1-15.3.3)** from *Elmasri & Navathe, Chapter 15* 📌

🧩 15.3 Normalization of Relations

1. Purpose of Normalization

Normalization is a process proposed by **E.F. Codd (1972)** to improve the design of database relations.

It tests whether a relation schema satisfies certain **normal form conditions** and decomposes it if needed.

2. What Normalization Does

It is a **design by analysis** method that:

- Starts with an existing relation schema.
- Evaluates it against the criteria for each **normal form**.
- **Decomposes** it into smaller, better-structured relations if it fails the test.

The process reduces redundancy and avoids anomalies (insertion, deletion, and update).

3. Historical Development of Normal Forms

Normal Form	Introduced By	Based On
1NF, 2NF, 3NF	Codd (1972)	Functional Dependencies (FDs)
BCNF	Boyce & Codd (later)	Stronger version of 3NF
4NF	Later work	Multivalued Dependencies
5NF	Later work	Join Dependencies

4. Purpose of Normalization

The goals are:

1. **Minimize data redundancy**
2. **Minimize insertion, deletion, and update anomalies**

In essence, normalization is a **filtering process** to purify design quality.

5. What Normalization Provides

Normalization offers:

- A **formal framework** for analyzing relation schemas using **keys** and **FDs**.
 - A **sequence of tests** (for each normal form) that can be applied until a desired level of normalization is achieved.
-

6. Definition of Normal Form

*The **normal form** of a relation refers to the **highest normal form condition** that it satisfies.*

Example:

If a relation satisfies 1NF and 2NF but not 3NF → it is said to be in **Second Normal Form (2NF)**.

7. Beyond Individual Relations

Checking each relation separately for 3NF or BCNF **is not sufficient** for a good overall database design.

The **set of relations as a whole** must satisfy two crucial properties after decomposition:

Property	Meaning	Importance
Nonadditive (Lossless) Join Property	Ensures no spurious tuples are produced when decomposed relations are joined back.	Must always hold
Dependency Preservation Property	Ensures that every functional dependency is still represented in at least one of the decomposed relations.	Desirable but not mandatory

🟢 *Lossless join = **essential***

🟡 *Dependency preservation = **preferred**, but sometimes sacrificed*

(Details and algorithms are discussed in Chapter 16.)

🧠 15.3.2 Practical Use of Normal Forms

1. Real-World Application

- Normalization is usually applied to **existing database designs, legacy files, or schemas** inherited from older systems.
- It ensures the resulting designs are of **high quality** and have the desirable theoretical properties.

2. Common Practice

Although higher normal forms (4NF, 5NF) exist:

- Most **real-world databases** are normalized only up to **3NF** or **BCNF**, sometimes **4NF**.

- This is because **higher-order dependencies** (multivalued or join) are **rare and hard to detect** in practice.

3. Balancing Performance

Designers sometimes **intentionally stop** normalization at **2NF or 3NF** for **performance reasons**, such as:

- Reducing the number of joins.
- Improving query speed.

This tradeoff, however, brings back **some anomalies**.

4. Denormalization

***Denormalization** is the **reverse process** of normalization.*

Definition:

It is the process of **storing the join** of higher normal form relations as a **base relation** that exists in a **lower normal form**.

Used when:

- Speed or simplicity is prioritized.
 - Redundancy and anomalies are acceptable for performance.
-

15.3.3 Keys and Attributes Participating in Keys

Before defining normal forms, we revisit the concept of **keys**.

1. Superkey

*A **superkey** of a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes $S \subseteq R$ such that no two tuples in any legal relation state have the same value for S .*

In other words, **superkey = unique identifier**, but **not necessarily minimal**.

2. Key (or Candidate Key)

A **key** is a **minimal superkey**, meaning that if you remove any attribute, it will no longer be a superkey.

Example:

- For **EMPLOYEE(Ssn, Ename, Bdate)**
 - `{Ssn}` is a **key** (minimal unique identifier).
 - `{Ssn, Ename}` or `{Ssn, Ename, Bdate}` are **superkeys** (not minimal).

If a relation has multiple keys → each is a **candidate key**.

One is chosen as the **primary key**, others are **secondary keys**.

3. Prime and Nonprime Attributes

Type	Definition	Example
Prime Attribute	An attribute that is part of any candidate key .	<code>Ssn</code> , <code>Pnumber</code>
Nonprime Attribute	Not part of any candidate key.	<code>Ename</code> , <code>Hours</code>

Example (from WORKS_ON relation):

- Prime attributes:** `Ssn` , `Pnumber`
- Nonprime attributes:** others (like `Hours`)

4. Transition to Normal Forms

Codd proposed **1NF**, **2NF**, and **3NF** as successive steps:

- Each step removes a specific kind of **redundancy or anomaly**.
- By definition, a relation in 3NF automatically satisfies **1NF and 2NF**.

Normal Form	Focus	Problem It Solves
1NF	Atomicity of data	Removes repeating groups

Normal Form	Focus	Problem It Solves
2NF	Partial dependency	Removes dependency on part of a composite key
3NF	Transitive dependency	Removes indirect dependency through non-key attributes

✅ Summary Table

Concept	Meaning
Normalization	Process to improve relational design using FDs
Goal	Minimize redundancy and anomalies
Key property	Must ensure lossless join; should preserve dependencies
Common practice	Normalize up to 3NF or BCNF
Denormalization	Storing joined relations in a lower normal form for performance
Prime attribute	Appears in some candidate key
Nonprime attribute	Does not appear in any candidate key

Would you like me to continue with **the detailed definitions and examples of 1NF, 2NF, and 3NF (Section 15.3.4-15.3.6)** next?

08/11/2025, 12:16:58



15.3.4 First Normal Form

First normal form (1NF) is now considered to be part of the formal definition of a relation in the basic (flat) relational model; historically, it was defined to disallow multivalued attributes, composite attributes, and their combinations. It states that the domain of an attribute must include only atomic (simple, indivisible) values and that the value of any attribute in a tuple must be a single value from the domain of that attribute. Hence, 1NF disallows having a set of values, a tuple of values, or a combination of both as an attribute value for a single tuple. In other words, 1NF disallows relations within relations or relations as attribute values within tuples. The only attribute values permitted by 1NF are single atomic (or indivisible) values.

Consider the DEPARTMENT relation schema shown in Figure 15.1, whose primary key is Dnumber, and suppose that we extend it by including the Dlocations attribute as shown in Figure 15.9(a). We assume that each department can have a number of locations. The DEPARTMENT schema and a sample relation state are shown in Figure 15.9. As we can see, this is not in 1NF because Dlocations is not an atomic attribute, as illustrated by the first tuple in Figure 15.9(b). There are two ways we can look at the Dlocations attribute:

- The domain of Dlocations contains atomic values, but some tuples can have a set of these values. In this case, Dlocations is not functionally dependent on the primary key Dnumber.

520

Chapter 15 Basics of Functional Dependencies and

Normalization for Relational Databases

(a)

DEPARTMENT

Dname

Dnumber

Dmgr_ssn

(b)

DEPARTMENT

Dname

Research

Administration

Headquarters 1

Dnumber

5

4

Dmgr_ssn

333445555

987654321

Dlocations

Dlocations

{Bellaire, Sugarland, Houston}

{Stafford}

888665555

(c)

{Houston}

DEPARTMENT

Figure 15.9

Normalization into 1NF. (a) A relation schema that is not in

1NF. (b) Sample state of relation DEPARTMENT. (c)

1NF version of the same relation with redundancy.

Dname

Research

Research

Research

Administration

Headquarters

Dnumber

5

5
 Dmgr_ssn
 333445555
 333445555
 Dlocation
 5
 4
 1
 333445555
 987654321
 888665555
 Bellaire
 Sugarland
 Houston
 Stafford
 Houston

■ The domain of Dlocations contains sets of values and hence is nonatomic. In this case, Dnumber $\rightarrow$ Dlocations because each set is considered a single member of the attribute domain.⁹

In either case, the DEPARTMENT relation in Figure 15.9 is not in 1NF; in fact, it does not even qualify as a relation according to our definition of relation in Section 3.1.

There are three main techniques to achieve first normal form for such a relation:

1. Remove the attribute Dlocations that violates 1NF and place it in a separate relation DEPT_LOCATIONS along with the primary key Dnumber of

DEPARTMENT. The primary key of this relation is the combination

{Dnumber, Dlocation}, as shown in Figure 15.2. A distinct tuple in

DEPT_LOCATIONS exists for each location of a department. This decomposes the non-1NF relation into two 1NF relations.

⁹In this case we can consider the domain of Dlocations to be the power set of the set of single locations; that is, the domain is made up of all possible

subsets of the set of single locations.

15.3 Normal Forms Based on Primary Keys

521

2. Expand the key so that there will be a separate tuple in the original

DEPARTMENT relation for each location of a DEPARTMENT, as shown in

Figure 15.9(c). In this case, the primary key becomes the combination

{Dnumber, Dlocation}. This solution has the disadvantage of introducing redundancy in the relation.

3. If a maximum number of values is known for the attribute—for example, if it

is known that at most three locations can exist for a department—replace the

Dlocations attribute by three atomic attributes: Dlocation1, Dlocation2, and

Dlocation3. This solution has the disadvantage of introducing NULL values if

most departments have fewer than three locations. It further introduces spu

rious semantics about the ordering among the location values that is not

originally intended. Querying on this attribute becomes more difficult; for

example, consider how you would write the query: List the departments that

have 'Bellaire' as one of their locations in this design.

Of the three solutions above, the first is generally considered best because it does

not suffer from redundancy and it is completely general, having no limit placed on

a maximum number of values. In fact, if we choose the second solution, it will be

decomposed further during subsequent normalization steps into the first solution.

First normal form also disallows multivalued attributes that are themselves com

posite. These are called nested relations because each tuple can have a relation

within it. Figure 15.10 shows how the EMP_PROJ relation could appear if nesting is allowed. Each tuple represents an employee entity, and a relation PROJS(Pnumber, Hours) within each tuple represents the employee's projects and the hours per week that employee works on each project. The schema of this EMP_PROJ relation can be represented as follows:

EMP_PROJ(Ssn, Ename, {PROJS(Pnumber, Hours)})

The set braces { } identify the attribute PROJS as multivalued, and we list the component attributes that form PROJS between parentheses (). Interestingly, recent trends for supporting complex objects (see Chapter 11) and XML data (see Chapter 12) attempt to allow and formalize nested relations within relational database systems, which were disallowed early on by 1NF.

Notice that Ssn is the primary key of the EMP_PROJ relation in Figures 15.10(a) and

(b), while Pnumber is the partial key of the nested relation; that is, within each tuple,

the nested relation must have unique values of Pnumber. To normalize this into 1NF,

we remove the nested relation attributes into a new relation and propagate the pri

mary key into it; the primary key of the new relation will combine the partial key

with the primary key of the original relation.

Decomposition and primary key

propagation yield the schemas EMP_PROJ1 and

EMP_PROJ2, as shown in Figure

15.10(c).

This procedure can be applied recursively to a relation with multiple-level nesting

to unnest the relation into a set of 1NF relations. This is useful in converting an

unnormalized relation schema with many levels of nesting into 1NF relations. The

522 Chapter 15 Basics of Functional Dependencies and

Normalization for Relational Databases

EMP_PROJ

(a)

Projs

Pnumber Hours Ssn Ename

EMP_PROJ1

(c)

Ssn Ename

EMP_PROJ2

Hours Ssn Pnumber

EMP_PROJ

(b)

Ssn

123456789

666884444

453453453

333445555

999887777

987987987

987654321

888665555

Zelaya, Alicia J.

Jabbar, Ahmad V.

Wallace, Jennifer S.

Borg, James E.

32.5

7.5

40.0

20.0

20.0

10.0

10.0

10.0

10.0

30.0

10.0

35.0

5.0

20.0

15.0

NULL

English, Joyce A.
Narayan, Ramesh K.
Smith, John B.
Wong, Franklin T.

Ename

3

1

2

1

2

2

20

3

10

30

10

10

20

30

30

20

Pnumber Hours

Figure 15.10

Normalizing nested relations into 1NF. (a)

Schema of the EMP_PROJ relation with a nested relation attribute PROJS. (b) Sample extension of the

EMP_PROJ relation showing nested relations within each tuple. (c)

Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

existence of more than one multivalued attribute in one relation must be handled carefully. As an example, consider the following non-1NF

relation:

PERSON (Ss#,{Car_lic#},{Phone#})

This relation represents the fact that a person has multiple cars and multiple phones. If strategy 2 above is followed, it results in an all-key relation:

PERSON_IN_1NF (Ss#, Car_lic#, Phone#)

15.3 Normal Forms Based on Primary Keys

523

To avoid introducing any extraneous relationship between Car_lic# and Phone#, all possible combinations of values are represented for every Ss#, giving rise to redundancy. This leads to the problems handled by multivalued dependencies and 4NF, which we will discuss in Section 15.6. The right way to deal with the two multivalued attributes in PERSON shown previously is to decompose it into two separate relations, using strategy 1 discussed above: P1(Ss#, Car_lic#) and P2(Ss#, Phone#).

15.3.5 Second Normal Form

Second normal form (2NF) is based on the concept of full functional dependency. A

functional dependency $X \rightarrow Y$ is a full functional dependency if removal of any attribute A from X means that the dependency does not hold any more; that is, for

any attribute $A \in X$, $(X - \{A\})$ does not functionally determine Y. A functional

dependency $X \rightarrow Y$ is a partial dependency if some attribute $A \in X$ can be removed

from X and the dependency still holds; that is, for some $A \in X$, $(X - \{A\}) \rightarrow Y$. In

Figure 15.3(b), $\{Ssn, Pnumber\} \rightarrow Hours$ is a full dependency (neither $Ssn \rightarrow Hours$ nor $Pnumber \rightarrow Hours$ holds). However, the dependency $\{Ssn, Pnumber\} \rightarrow Ename$ is partial because $Ssn \rightarrow Ename$ holds.

Definition. A relation schema R is in 2NF if every nonprime attribute A in R is

fully functionally dependent on the primary key of R. The test for 2NF involves testing for functional dependencies whose left-hand side attributes are part of the primary key. If the primary key contains a single attribute, the test need not be applied at all. The EMP_PROJ relation in Figure 15.3(b) is in 1NF but is not in 2NF. The nonprime attribute Ename violates 2NF because of FD2, as do the nonprime attributes Pname and Plocation because of FD3. The functional dependencies FD2 and FD3 make Ename, Pname, and Plocation partially dependent on the primary key {Ssn, Pnumber} of EMP_PROJ, thus violating the 2NF test.

If a relation schema is not in 2NF, it can be second normalized or 2NF normalized into a number of 2NF relations in which nonprime attributes are associated only with the part of the primary key on which they are fully functionally dependent.

Therefore, the functional dependencies FD1, FD2, and FD3 in Figure 15.3(b) lead to the decomposition of EMP_PROJ into the three relation schemas EP1, EP2, and EP3 shown in Figure 15.11(a), each of which is in 2NF.

15.3.6 Third Normal Form

Third normal form (3NF) is based on the concept of transitive dependency.^A

functional dependency $X \rightarrow Y$ in a relation schema R is a transitive dependency if

there exists a set of attributes Z in R that is neither a candidate key nor a subset of

any key of R,¹⁰ and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold. The dependency $Ssn \rightarrow Dmgr_ssn$

is transitive through Dnumber in EMP_DEPT in Figure 15.3(a), because both the

¹⁰This is the general definition of transitive dependency. Because we are concerned only with primary

keys in this section, we allow transitive dependencies

where X is the primary key but Z may be (a subset of) a candidate key.

524

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

(a)

EMP_PROJ

Pnumber Hours Ename Pname Plocation

Ssn

FD1

FD2

FD3

2NF Normalization

EP1

Ssn

FD1

Pnumber Hours

EP2

Ssn

Ename

FD2

EP3

Pnumber

FD3

Pname

Plocation

(b)

EMP_DEPT

Ename

Ssn

Bdate Address Dnumber Dname Dmgr_ssn

3NF Normalization

ED1

Ename

Ssn

Bdate Address Dnumber

ED2

Dnumber

Dname Dmgr_ssn

Figure 15.11

Normalizing into 2NF and 3NF. (a) Normalizing

EMP_PROJ into

2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.

dependencies $Ssn \rightarrow Dnumber$ and $Dnumber \rightarrow Dmgr_ssn$ hold and $Dnumber$ is neither a key itself nor a subset of the key of EMP_DEPT. Intuitively, we can see that

the dependency of $Dmgr_ssn$ on $Dnumber$ is undesirable in EMP_DEPT since

$Dnumber$ is not a key of EMP_DEPT.

Definition. According to Codd's original definition, a relation schema R is in

3NF if it satisfies 2NF and no nonprime attribute of R is transitively dependent on the primary key.

The relation schema EMP_DEPT in Figure 15.3(a) is in 2NF, since no partial dependencies on a key exist. However, EMP_DEPT is not in 3NF because of the transitive dependency of $Dmgr_ssn$ (and also $Dname$) on Ssn via $Dnumber$. We can normalize

15.4 General Definitions of Second and Third Normal Forms

525

EMP_DEPT by decomposing it into the two 3NF relation schemas ED1 and ED2

shown in Figure 15.11(b). Intuitively, we see that ED1 and ED2 represent independent

entity facts about employees and departments. A NATURAL JOIN operation on

ED1 and ED2 will recover the original relation EMP_DEPT without generating spurious tuples.

Intuitively, we can see that any functional dependency in which the left-hand side is

part (a proper subset) of the primary key, or any functional dependency in which

the left-hand side is a nonkey attribute, is a problematic FD. 2NF and 3NF normal

ization remove these problem FDs by decomposing the original relation into new

relations. In terms of the normalization process, it is not necessary to remove the partial dependencies before the transitive dependencies, but historically, 3NF has been defined with the assumption that a relation is tested for 2NF first before it is tested for 3NF. Table 15.1 informally summarizes the three normal forms based on primary keys, the tests used in each case, and the corresponding remedy or normalization performed to achieve the normal form

08/11/2025, 12:30:01



Excellent — you've just covered **Sections 15.3.4 to 15.3.6** of *Elmasri & Navathe's Database Systems* chapter on **Normalization**.

Let's summarize and clarify these three normal forms with key ideas and examples from your excerpt 📌

🧩 First Normal Form (1NF)

Definition

A relation is in **1NF** if **every attribute** contains **only atomic (indivisible) values** — that is, **no sets, arrays, or nested records**.

Violations

Occurs when:

- An attribute has **multiple values** (like `{Bellaire, Sugarland, Houston}` in `Dlocations`)
- An attribute is **composite or nested**

Ways to Fix

1. **Decompose** the multivalued attribute into a separate relation:

```
DEPT_LOCATIONS(Dnumber, Dlocation)
```

✅ *Best method — eliminates redundancy.*

2. **Expand the key** so each multivalued becomes a separate tuple:

```
DEPARTMENT(Dnumber, Dlocation, Dname, Dmgr_ssn)
```

⚠️ *Causes redundancy.*

3. **Split into fixed atomic columns** (e.g., Dlocation1, Dlocation2, Dlocation3)

⚠️ *Leads to NULLs and meaningless ordering.*

Example:

Not in 1NF:

```
DEPARTMENT(Dnumber, Dname, Dmgr_ssn, Dlocations)
5, Research, 333445555, {Bellaire, Sugarland, Houston}
```

After 1NF (using decomposition):

```
DEPARTMENT(Dnumber, Dname, Dmgr_ssn)
DEPT_LOCATIONS(Dnumber, Dlocation)
```

🧠 Second Normal Form (2NF)

Definition

A relation is in **2NF** if:

- It is in **1NF**
- Every **non-prime attribute** (not part of a candidate key) is **fully functionally dependent** on the **entire primary key**

Violation

A **partial dependency** exists — i.e., a non-key attribute depends on part of a composite key.

Fix

- **Decompose** the relation so that each non-prime attribute depends on a full key.

Example:

From Figure 15.3(b):

```
EMP_PROJ(Ssn, Pnumber, Hours, Ename, Pname, Plocation)
```

Key: {Ssn, Pnumber}

Partial Dependencies:

- Ssn → Ename
- Pnumber → Pname, Plocation

Decomposition (2NF):

```
EP1(Ssn, Pnumber, Hours)
EP2(Ssn, Ename)
EP3(Pnumber, Pname, Plocation)
```

Third Normal Form (3NF)

Definition

A relation is in **3NF** if:

- It is in **2NF**, and
- **No non-prime attribute** is **transitively dependent** on the **primary key**.

Violation

A **transitive dependency** occurs when:

```
Key → Non-key1 → Non-key2
```

Example:

```
Ssn → Dnumber and Dnumber → Dmgr_ssn
```

⇒ Transitive dependency: Ssn → Dmgr_ssn

Fix

- **Decompose** to remove the transitive dependency.

Example:

```
EMP_DEPT(Ssn, Ename, Bdate, Address, Dnumber, Dname, Dmgr_ssn)
```

Problem:

Ssn → Dnumber and Dnumber → Dmgr_ssn

3NF Decomposition:

```
ED1(Ssn, Ename, Bdate, Address, Dnumber)
```

```
ED2(Dnumber, Dname, Dmgr_ssn)
```

✅ Removes transitive dependency, preserves information, and avoids anomalies.

Summary Table

Normal Form	Based On	Violation	Fix (Normalization)
1NF	Atomicity of values	Multivalued or nested attributes	Remove multivalued attributes into new relation
2NF	Full functional dependency	Partial dependency (non-key depends on part of composite key)	Decompose so non-prime attributes depend on full key
3NF	Transitive dependency	Non-prime depends transitively on primary key	Decompose to remove transitive dependencies

Would you like me to continue this summary to include **Boyce-Codd Normal Form (BCNF)** next (Section 15.3.7)? It's the next logical step after 3NF and helps formalize these ideas further.

08/11/2025, 12:30:09



General Definitions of Second and Third Normal Forms

In general, we want to design our relation schemas so that they have neither partial nor transitive dependencies because these types of dependencies cause the update anomalies discussed in Section 15.1.2. The steps for normalization into 3NF relations that we have discussed so far disallow partial and transitive dependencies on the primary key. The normalization procedure described so far is useful for analysis in practical situations for a given database where primary keys have already been defined. These definitions, however, do not take other candidate keys of a relation, if

Table 15.1 Summary of Normal Forms Based on Primary Keys and Corresponding Normalization

Normal Form

First (1NF)

Second (2NF)

Test

Relation should have no multivalued attributes or nested relations.

For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.
--

Remedy (Normalization)

Form new relations for each multivalued attribute or nested relation.

Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
--

Third (3NF)

Relation should not have a nonkey attribute functionally determined by
--

another nonkey attribute (or by a set of nonkey attributes). That is, there should be no transitive dependency of a nonkey attribute on the primary key.

Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).

526

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

any, into account. In this section we give the more general definitions of 2NF and

3NF that take all candidate keys of a relation into account. Notice that this does not

affect the definition of 1NF since it is independent of keys and functional dependencies.

As a general definition of prime attribute, an attribute that is part of any

candidate key will be considered as prime. Partial and full functional dependencies

and transitive dependencies will now be considered with respect to all candidate keys

of a relation.

15.4.1 General Definition of Second Normal Form

Definition. A relation schema R is in second normal form (2NF) if every non

prime attribute A in R is not partially dependent on any key of R .¹¹

The test for 2NF involves testing for functional dependencies whose left-hand side

attributes are part of the primary key. If the primary key contains a single attribute,

the test need not be applied at all. Consider the relation schema $LOTS$ shown in

Figure 15.12(a), which describes parcels of land for sale in various counties of a

state. Suppose that there are two candidate keys: $Property_id\#$ and $\{County_name,$

$Lot\#\}$; that is, lot numbers are unique only within each county, but $Property_id\#$

numbers are unique across counties for the entire state.

Based on the two candidate keys `Property_id#` and `{County_name, Lot#}`, the functional dependencies `FD1` and `FD2` in Figure 15.12(a) hold. We choose `Property_id#` as the primary key, so it is underlined in Figure 15.12(a), but no special consideration will be given to this key over the other candidate key. Suppose that the following two additional functional dependencies hold in `LOTS`:

`FD3: County_name → Tax_rate`

`FD4: Area → Price`

In words, the dependency `FD3` says that the tax rate is fixed for a given county (does not vary lot by lot within the same county), while `FD4` says that the price of a lot is determined by its area regardless of which county it is in. (Assume that this is the price of the lot for tax purposes.)

The `LOTS` relation schema violates the general definition of 2NF because `Tax_rate` is partially dependent on the candidate key `{County_name, Lot#}`, due to `FD3`. To normalize `LOTS` into 2NF, we decompose it into the two relations `LOTS1` and `LOTS2`,

shown in Figure 15.12(b). We construct `LOTS1` by removing the attribute `Tax_rate` that violates 2NF from `LOTS` and placing it with `County_name` (the left-hand side of `FD3` that causes the partial dependency) into another relation `LOTS2`. Both `LOTS1` and `LOTS2` are in 2NF. Notice that `FD4` does not violate 2NF and is carried over to `LOTS1`.

This definition can be restated as follows: A relation schema `R` is in 2NF if every nonprime attribute `A` in `R` is fully functionally dependent on every key of `R`.

15.4 General Definitions of Second and Third Normal Forms

Figure 15.12

Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies

FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c)

Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Summary of the progressive normalization of LOTS.

Candidate Key

(a)

(b)

LOTS

Property_id#

FD1

FD2

FD3

FD4

County_name

Lot# Area Price Tax_rate

LOTS1

Property_id#

County_name

Lot# Area Price

FD1

LOTS2

County_name

FD3

Tax_rate

FD2

FD4

(c)

(d)

LOTS1A

Property_id#

County_name

FD1

FD2

LOTS1

LOTS1A

LOTS1B

Lot# Area

LOTS
LOTS1B
LOTS2
LOTS2
Area Price
FD4
1NF
2NF
3NF
528

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

15.4.2 General Definition of Third Normal Form

Definition. A relation schema R is in third normal form (3NF) if, whenever a

nontrivial functional dependency $X \rightarrow A$ holds in R , either (a) X is a superkey of R , or (b) A is a prime attribute of R .

According to this definition, LOTS2 (Figure 15.12(b)) is in 3NF. However, FD4 in

LOTS1 violates 3NF because Area is not a superkey and Price is not a prime attribute in LOTS1. To normalize LOTS1 into 3NF, we decompose it into the relation schemas

LOTS1A and LOTS1B shown in Figure 15.12(c). We construct LOTS1A by removing the attribute Price that violates 3NF from LOTS1 and placing it with Area (the left hand side of FD4 that causes the transitive dependency) into another relation

LOTS1B. Both LOTS1A and LOTS1B are in 3NF.

Two points are worth noting about this example and the general definition of 3NF:

■ LOTS1 violates 3NF because Price is transitively dependent on each of the candidate keys of LOTS1 via the nonprime attribute Area.

■ This general definition can be applied directly to test whether a relation schema is in 3NF; it does not have to go through 2NF first. If we apply the

above 3NF definition to LOTS with the dependencies FD1 through FD4, we find that both FD3 and FD4 violate 3NF. Therefore, we could decompose LOTS into LOTS1A, LOTS1B, and LOTS2 directly. Hence, the transitive and partial dependencies that violate 3NF can be removed in any order.

15.4.3 Interpreting the General Definition of Third Normal Form

A relation schema R violates the general definition of 3NF if a functional dependency $X \rightarrow A$ holds in R that does not meet either condition—meaning that it violates both conditions (a) and (b) of 3NF. This can occur due to two types of problematic functional dependencies:

- A nonprime attribute determines another nonprime attribute. Here we typically have a transitive dependency that violates 3NF.

- A proper subset of a key of R functionally determines a nonprime attribute.

Here we have a partial dependency that violates 3NF (and also 2NF).

Therefore, we can state a general alternative definition of 3NF as follows:

Alternative Definition. A relation schema R is in 3NF if every nonprime attribute of R meets both of the following conditions:

- It is fully functionally dependent on every key of R .
- It is nontransitively dependent on every key of R .

08/11/2025, 12:45:31



Excellent — this section you've pasted covers the **formal (general) definitions** of **Second Normal Form (2NF)** and **Third Normal Form (3NF)**, extending beyond the simpler primary-key-based versions.

Let's summarize and interpret this passage carefully so you can fully grasp the *general theory* of normalization.

Recap: Why General Definitions Are Needed

Earlier, 2NF and 3NF were defined *only with respect to the primary key*.

But in reality, a relation can have **multiple candidate keys** (alternative unique identifiers).

So, the **general definitions** take into account:

- **All candidate keys**, not just the primary key.
- **All dependencies** involving any of them.

This prevents anomalies even when multiple keys exist.

◆ General Definition of 2NF

Definition:

A relation schema *R* is in **Second Normal Form (2NF)** if **every nonprime attribute** is **fully functionally dependent** on **every key** of *R*.

✓ Meaning:

- **Nonprime attribute** = any attribute that is *not part of any candidate key*.
- **Fully functionally dependent** = no part of a key alone should determine the attribute.
In other words, **no partial dependency** on a key should exist.

💡 Example: LOTS relation

```
LOTS(Property_id#, County_name, Lot#, Area, Price, Tax_rate)
```

Candidate Keys:

- `Property_id#`
- `{County_name, Lot#}`

Functional Dependencies:

1. FD1: $\text{Property_id\#} \rightarrow \text{County_name}, \text{Lot\#}, \text{Area}, \text{Price}, \text{Tax_rate}$
2. FD2: $\{\text{County_name}, \text{Lot\#}\} \rightarrow \text{Property_id\#}, \text{Area}, \text{Price}, \text{Tax_rate}$
3. FD3: $\text{County_name} \rightarrow \text{Tax_rate}$
4. FD4: $\text{Area} \rightarrow \text{Price}$

Now:

- FD3 violates 2NF because `Tax_rate` depends only on `County_name` (part of a composite key `\{\text{County\_name}, \text{Lot\#}\}`).

So, decompose:

```
LOTS1(Property_id#, County_name, Lot#, Area, Price)
LOTS2(County_name, Tax_rate)
```

Now both are in **2NF**.

◆ General Definition of 3NF

Definition:

A relation schema R is in **Third Normal Form (3NF)** if, for every nontrivial functional dependency $X \rightarrow A$ in R ,
either

1. X is a **superkey**, or
2. A is a **prime attribute** (part of some candidate key).

✓ Meaning:

3NF removes **transitive dependencies**, but allows dependencies where:

- The determinant is a key (that's fine).
- The dependent is part of a key (also fine).

💡 Example continued: LOTS1

From earlier:

```
LOTS1(Property_id#, County_name, Lot#, Area, Price)
```

FD4: Area → Price

Here:

- `Area` is not a key or superkey.
- `Price` is not a prime attribute.

So FD4 **violates 3NF**.

Hence, we decompose further:

```
LOTS1A(Property_id#, County_name, Lot#, Area)
LOTS1B(Area, Price)
```

Now:

- `LOTS1A` and `LOTS1B` are both in **3NF**.

◆ Alternative Definition (Simplified Summary)

A relation is in **3NF** if **every nonprime attribute** is:

1. **Fully functionally dependent** on **every key**, and
2. **Nontransitively dependent** on **every key**.

So 3NF = 2NF + No transitive dependencies.

🧠 Conceptual Summary Table

Normal Form	What It Removes	Key Idea	Example Problem Fixed
1NF	Repeating groups / multivalued attributes	All attribute values are atomic	Departments with <code>{Houston, Bellaire}</code> in one field
2NF	Partial dependencies	Nonprime attributes must depend on the	Ename depending on <code>Ssn</code> only

Normal Form	What It Removes	Key Idea	Example Problem Fixed
		<i>whole</i> key	
3NF	Transitive dependencies	Nonprime attributes must depend <i>directly</i> on the key	Dmgr_ssn depending on Dnumber instead of Ssn

✓ In summary

- **1NF:** Every attribute is atomic.
- **2NF:** No partial dependency (depends on full key).
- **3NF:** No transitive dependency (depends only on the key, not another nonkey).
- **General definitions:** Extend the same logic to all **candidate keys**, not just the primary key.

Would you like me to include a **diagrammatic summary** (like the progression from LOTS → LOTS1 → LOTS1A, LOTS1B, LOTS2) in text form for easy memorization?

08/11/2025, 12:45:36



Boyce-Codd Normal Form

Boyce-Codd normal form (BCNF) was proposed as a simpler form of 3NF, but it

was found to be stricter than 3NF. That is, every relation in BCNF is also in 3NF;

however, a relation in 3NF is not necessarily in BCNF.

Intuitively, we can see the need

for a stronger normal form than 3NF by going back to the LOTS relation schema in

Figure 15.12(a) with its four functional dependencies FD1 through FD4. Suppose

that we have thousands of lots in the relation but the lots are from only two coun

ties: DeKalb and Fulton. Suppose also that lot sizes in DeKalb County are only 0.5,

0.6, 0.7, 0.8, 0.9, and 1.0 acres, whereas lot sizes in Fulton County are restricted to

1.1, 1.2, ..., 1.9, and 2.0 acres. In such a situation we would have the additional func

tional dependency FD5: $\text{Area} \rightarrow \text{County_name}$. If we add this to the other dependen

cies, the relation schema LOTS1A still is in 3NF because County_name is a prime

attribute.

The area of a lot that determines the county, as specified by FD5, can be represented

by 16 tuples in a separate relation $R(\text{Area}, \text{County_name})$, since there are only 16 pos

sible Area values (see Figure 15.13). This

representation reduces the redundancy of

repeating the same information in the thousands of LOTS1A tuples. BCNF is a

stronger normal form that would disallow LOTS1A and suggest the need for decom

posing it.

Definition. A relation schema R is in BCNF if whenever a nontrivial functional

dependency $X \rightarrow A$ holds in R , then X is a superkey of R .

(a)

(b)
LOTS1A
Property_id#
FD1
FD2
LOTS1AX
Property_id#
R
A

Figure 15.13
County_name Lot# Area
FD5

BCNF Normalization

LOTS1AY
Area Lot#

Area

Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF.

County_name

B C

FD1

FD2

530 Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

TEACH

Student

Narayan

Smith

Smith

Smith

Mark

Navathe

Ammar

Schulman

Operating Systems

Database

Database

Theory

Wallace

Wallace

Wong

Zelaya

Mark

Ahamad

Omiecinski

Navathe

Database

Database

Operating Systems

Database

Course Instructor

Narayan Operating Systems Ammar

Figure 15.14

A relation TEACH that is in 3NF but not BCNF.

The formal definition of BCNF differs from the definition of 3NF in that condition

(b) of 3NF, which allows $A \rightarrow B$ to be prime, is absent from BCNF. That makes BCNF a

stronger normal form compared to 3NF. In our example, FD5 violates BCNF in

LOTS1A because AREA is not a superkey of LOTS1A. Note that FD5 satisfies 3NF in

LOTS1A because County_name is a prime attribute (condition b), but this condition

does not exist in the definition of BCNF. We can decompose LOTS1A into two BCNF

relations LOTS1AX and LOTS1AY, shown in Figure 15.13(a). This decomposition

loses the functional dependency FD2 because its attributes no longer coexist in the same relation after decomposition.

In practice, most relation schemas that are in 3NF are also in BCNF. Only if $X \rightarrow A$

holds in a relation schema R with X not being a superkey and A being a prime

attribute will R be in 3NF but not in BCNF. The relation schema R shown in Figure

15.13(b) illustrates the general case of such a relation. Ideally, relational database design should strive to achieve BCNF or 3NF for every relation schema. Achieving the normalization status of just 1NF or 2NF is not considered adequate, since they were developed historically as stepping stones to 3NF and BCNF.

As another example, consider Figure 15.14, which shows a relation TEACH with the following dependencies:

FD1: {Student, Course} → Instructor

FD2: 12 Instructor → Course

Note that {Student, Course} is a candidate key for this relation and that the dependencies shown follow the pattern in Figure 15.13(b), with Student as A, Course as B, and Instructor as C. Hence this relation is in 3NF but not BCNF. Decomposition of

this relation schema into two schemas is not straightforward because it may be

12 This dependency means that each instructor teaches one course is a constraint for this application.

15.6 Multivalued Dependency and Fourth Normal Form
531

decomposed into one of the three following possible pairs:

1. {Student, Instructor} and {Student, Course}.
2. {Course, Instructor} and {Course, Student}.
3. {Instructor, Course} and {Instructor, Student}.

All three decompositions lose the functional dependency FD1. The desirable decomposition of those just shown is 3 because it will not generate spurious tuples after a join.

A test to determine whether a decomposition is nonadditive (or lossless) is discussed in Section 16.2.4 under Property NJB. In general, a relation not in BCNF should be decomposed so as to meet this property. We make sure that we meet this property, because

nonadditive decomposition is a must during normalization. We may have to possibly forgo the preservation of all functional dependencies in the decomposed relations, as is the case in this example. Algorithm 16.5 does that and could be used above to give decomposition 3 for TEACH, which yields two relations in BCNF as: (Instructor, Course) and (Instructor, Student). Note that if we designate (Student, Instructor) as a primary key of the relation TEACH, the FD $\text{Instructor} \rightarrow \text{Course}$ causes a partial (non-full-functional) dependency of Course on a part of this key. This FD may be removed as a part of second normalization yielding exactly the same two relations in the result. This is an example of a case where we may reach the same ultimate BCNF design via alternate paths of normalization.

08/11/2025, 13:03:44



Excellent — this passage discusses the **Boyce-Codd Normal Form (BCNF)** and how it refines the concepts of **Third Normal Form (3NF)**.

Here's a **clear breakdown and explanation** of everything in that section:

◆ Why BCNF Was Introduced

3NF allows certain anomalies to remain because it **permits a dependency**

$X \rightarrow A$ **even when X is not a superkey**, as long as A is a **prime attribute** (i.e., part of some candidate key).

BCNF was created to **remove even those cases**.

It's stricter — every relation in BCNF is also in 3NF, but **not every 3NF relation is in BCNF**.

◆ Example — The LOTS Relation

The LOTS example from earlier (Figure 15.12(a)) had these functional dependencies (FDs):

FD	Dependency	Meaning
FD1	$\text{Property_id\#} \rightarrow \text{County_name}, \text{Lot\#}, \text{Area}, \text{Price}, \text{Tax_rate}$	Property ID identifies a lot
FD2	$\{\text{County_name}, \text{Lot\#}\} \rightarrow \text{Property_id\#}$	Lot number unique only within county
FD3	$\text{County_name} \rightarrow \text{Tax_rate}$	County determines tax rate
FD4	$\text{Area} \rightarrow \text{Price}$	Price determined by lot area

After normalization up to **3NF**, everything looked fine.

◆ But Then We Add Another Dependency

Now suppose a new dependency arises:

FD5: $\text{Area} \rightarrow \text{County_name}$

Because in practice, certain *area ranges* only exist in certain counties (e.g., DeKalb lots are 0.5–1.0 acres, Fulton lots are 1.1–2.0 acres).

◆ Why This Breaks BCNF

- In **LOTS1A**, the dependency $\text{Area} \rightarrow \text{County_name}$ holds.
- But here, **Area** is **not a superkey** of LOTS1A.
- Thus, **FD5 violates BCNF**, even though it still satisfies 3NF (because **County_name** is a prime attribute).

◆ BCNF Definition

A relation schema **R** is in **Boyce-Codd Normal Form (BCNF)** if whenever a nontrivial functional dependency $X \rightarrow A$ holds in **R**, **X is a superkey of R**.

◆ BCNF vs 3NF

Concept	3NF Condition	BCNF Condition
Dependency rule	If $X \rightarrow A$, then either X is a superkey or A is a prime attribute	If $X \rightarrow A$, then X must be a superkey
Strictness	Less strict	More strict
Inclusion	All BCNF relations are in 3NF	Not all 3NF relations are in BCNF

◆ LOTS1A BCNF Decomposition

Because of FD5 (`Area → County_name`), LOTS1A should be decomposed:

- **LOTS1AX(Property_id#, Lot#, Area)**
- **R(Area, County_name)**

This eliminates redundancy (each area-county pair stored once).

However, this decomposition **loses FD2** (`{County_name, Lot#} → Property_id#`).

👉 *This shows a tradeoff: achieving BCNF can cause the **loss of some functional dependencies**, though it ensures **no redundancy or anomalies**.*

◆ General Case Example (Figure 15.13b)

Consider a generic relation **R(A, B, C)** with:

FD1: `{A, B} → C`
FD2: `C → B`

- `{A, B}` is a key $\Rightarrow$ 3NF satisfied.
- But `C → B` violates BCNF (since C is not a superkey).

Thus, **R is in 3NF but not in BCNF**.

◆ Another Example — The TEACH Relation

TEACH(Student, Course, Instructor)

Functional dependencies:

FD1: {Student, Course} → Instructor

FD2: Instructor → Course

- {Student, Course} is a candidate key.
- So the relation is in **3NF**.
- But it's **not in BCNF**, because in FD2, Instructor is **not a superkey** of TEACH.

◆ BCNF Decomposition of TEACH

Three possible decompositions:

1. {Student, Instructor} and {Student, Course}
2. {Course, Instructor} and {Course, Student}
3. **{Instructor, Course} and {Instructor, Student}** ✓ (Best — avoids spurious tuples)

The third one is the **nonadditive (lossless)** decomposition that preserves meaning.

◆ Key Takeaways

- **3NF** removes *most* redundancy (partial & transitive dependencies).
- **BCNF** removes *all* redundancy due to functional dependencies.
- **BCNF rule:** *Every determinant must be a candidate key.*
- **Tradeoff:** Sometimes, BCNF decomposition may lose dependency preservation.

Would you like me to give a **side-by-side example table** showing how 2NF → 3NF → BCNF normalization works step-by-step (using the LOTS or TEACH example)? It visually clarifies how each form removes a specific kind of dependency.

08/11/2025, 13:03:48



Multivalued Dependency and Fourth Normal Form

So far we have discussed the concept of functional dependency, which is by far the most important type of dependency in relational database design theory, and normal forms based on functional dependencies. However, in many cases relations have constraints that cannot be specified as functional dependencies. In this section, we discuss the concept of multivalued dependency (MVD) and define fourth normal form, which is based on this dependency. A more formal discussion of MVDs and their properties is deferred to Chapter 16. Multivalued dependencies are a consequence of first normal form (1NF) (see Section 15.3.4), which disallows an attribute in a tuple to have a set of values, and the accompanying process of converting an unnormalized relation into 1NF. If we have two or more multivalued independent attributes in the same relation schema, we get into a problem of having to repeat every value of one of the attributes with every value of the other attribute to keep the relation state consistent and to maintain the independence among the attributes involved. This constraint is specified by a multivalued dependency.

532

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

For example, consider the relation EMP shown in Figure 15.15(a). A tuple in this

EMP relation represents the fact that an employee whose name is Ename works on the project whose name is Pname and has a dependent whose name is Dname. An

employee may work on several projects and may have several dependents, and the employee's projects and dependents are independent of one another.¹³ To keep the relation state consistent, and to avoid any spurious relationship between the two independent attributes, we must have a separate tuple to represent every combination of an employee's dependent and an employee's project. This constraint is spec

Figure 15.15

Fourth and fifth normal forms.

(a) The EMP relation with two MVDs: $\text{Ename} \twoheadrightarrow \text{Pname}$ and $\text{Ename} \twoheadrightarrow \text{Dname}$.

(b) Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS.

(c) The relation SUPPLY with no MVDs is in 4NF but not in 5NF if it has the JD(R1, R2, R3).

(d) Decomposing the relation SUPPLY into the 5NF relations R1, R2, R3.

(a) EMP

Ename

Smith

Smith

Smith

Smith

(b) EMP_PROJECTS

Ename

Smith

Smith

Pname

X

Y

(c)

Pname

Y

X

Y

X

Dname

John
Anna
Anna
John
EMP_DEPENDENTS
Dname
Ename
Smith
Smith
SUPPLY
Sname
Smith
Smith
Adamsky
Walton
Adamsky
Adamsky
Smith
Part_name
Bolt
Nut
Bolt
Nut
Nail
Bolt
Bolt
Proj_name
ProjX
ProjY
ProjY
ProjZ
ProjX
ProjX
ProjY
John
Anna
(d) R1
Sname
Smith
Smith
Adamsky

Walton
Adamsky
Part_name
Bolt
Nut
Bolt
Nut
Nail
R2
Sname
Smith
Smith
Adamsky
Walton
Adamsky
Proj_name
ProjX
ProjY
ProjY
ProjZ
ProjX
R3
Part_name
Bolt
Nut
Bolt
Nut
Nail
Proj_name
ProjX
ProjY
ProjY
ProjZ
ProjX

13 In an ER diagram, each would be represented as a multivalued attribute or as a weak entity type (see Chapter 7).

15.6 Multivalued Dependency and Fourth Normal Form

533

ified as a multivalued dependency on the EMP relation, which we define in this sec

tion. Informally, whenever two independent 1:N relationships A:B and A:C are mixed in the same relation, R(A, B, C), an MVD may arise.¹⁴

15.6.1 Formal Definition of Multivalued Dependency

Definition. A multivalued dependency $X \twoheadrightarrow Y$ specified on relation schema R, where X and Y are both subsets of R, specifies the following constraint on any relation state r of R: If two tuples t1 and t2 exist in r such that $t1[X] = t2[X]$, then two tuples t3 and t4 should also exist in r with the following properties,¹⁵ where

we use Z to denote $(R - (X \cup Y))$:¹⁶

- $t3[X] = t4[X] = t1[X] = t2[X]$.
- $t3[Y] = t1[Y]$ and $t4[Y] = t2[Y]$.
- $t3[Z] = t2[Z]$ and $t4[Z] = t1[Z]$.

Whenever $X \twoheadrightarrow Y$ holds, we say that X multidetermines Y. Because of the symme

try in the definition, whenever $X \twoheadrightarrow Y$ holds in R, so does $X \twoheadrightarrow Z$. Hence, $X \twoheadrightarrow Y$ implies $X \twoheadrightarrow Z$, and therefore it is sometimes written as $X \twoheadrightarrow Y|Z$.

An MVD $X \twoheadrightarrow Y$ in R is called a trivial MVD if (a) Y is a subset of X, or (b) $X \cup Y = R$. For example, the relation EMP_PROJECTS in Figure 15.15(b) has the trivial

MVD $Ename \twoheadrightarrow Pname$. An MVD that satisfies neither (a) nor (b) is called a

nontrivial MVD. A trivial MVD will hold in any relation state r of R; it is called trivial because it does not specify any significant or meaningful constraint on R.

If we have a nontrivial MVD in a relation, we may have to repeat values redundantly

in the tuples. In the EMP relation of Figure 15.15(a), the values 'X' and 'Y' of Pname

are repeated with each value of Dname (or, by symmetry, the values 'John' and 'Anna'

of Dname are repeated with each value of Pname).

This redundancy is clearly undesirable. However, the EMP schema is in BCNF because no functional dependencies hold in EMP. Therefore, we need to define a fourth normal form that is stronger than BCNF and disallows relation schemas such as EMP. Notice that relations containing nontrivial MVDs tend to be all-key relations—that is, their key is all their attributes taken together. Furthermore, it is rare that such all-key relations with a combinatorial occurrence of repeated values would be designed in practice.

However, recognition of MVDs as a potential problematic dependency is essential in relational design.

We now present the definition of fourth normal form (4NF), which is violated

when a relation has undesirable multivalued dependencies, and hence can be used to identify and decompose such relations.

14 This MVD is denoted as $A \twoheadrightarrow B|C$.

15 The tuples t_1 , t_2 , t_3 , and t_4 are not necessarily distinct.

16 Z is shorthand for the attributes in R after the attributes in $(X \cup Y)$ are removed from R .

534

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

Definition. A relation schema R is in 4NF with respect to a set of dependencies

F (that includes functional dependencies and multivalued dependencies) if, for

every nontrivial multivalued dependency $X \twoheadrightarrow Y$ in F , X is a superkey for R .

We can state the following points:

- An all-key relation is always in BCNF since it has no FDs.

- An all-key relation such as the EMP relation in Figure 15.15(a), which has no FDs but has the MVD $\text{Ename} \twoheadrightarrow \text{Pname} \mid \text{Dname}$, is

not in 4NF.

■ A relation that is not in 4NF due to a nontrivial MVD must be decomposed to convert it into a set of relations in 4NF.

■ The decomposition removes the redundancy caused by the MVD.

The process of normalizing a relation involving the nontrivial MVDs that is not in

4NF consists of decomposing it so that each MVD is represented by a separate rela

tion where it becomes a trivial MVD. Consider the EMP relation in Figure 15.15(a).

EMP is not in 4NF because in the nontrivial MVDs $Ename \rightarrow$

$\rightarrow Pname$ and $Ename$

$\rightarrow\rightarrow Dname$, and $Ename$ is not a superkey of EMP. We decompose EMP into

EMP_PROJECTS and EMP_DEPENDENTS, shown in Figure 15.15(b). Both

EMP_PROJECTS and EMP_DEPENDENTS are in 4NF, because the MVDs $Ename$

$\rightarrow\rightarrow Pname$ in EMP_PROJECTS and $Ename \rightarrow\rightarrow Dname$ in EMP_DEPENDENTS are

trivial MVDs. No other nontrivial MVDs hold in either EMP_PROJECTS or

EMP_DEPENDENTS. No FDs hold in these relation schemas either.

08/11/2025, 13:10:18



Excellent — this section moves beyond **functional dependencies (FDs)** and introduces a *new kind* of dependency called the **multivalued dependency (MVD)**, which leads to the **Fourth Normal Form (4NF)**.

Let's carefully unpack it step by step so you fully understand what MVDs are, why they arise, and how 4NF fixes the issues they create.



1 Why MVDs Are Needed

Up to **BCNF**, we handled anomalies caused by **functional dependencies (FDs)** — that is, cases where one attribute depends on another.

But sometimes, **no FDs exist**, yet redundancy still appears.

This happens when:

- We have **two or more independent multivalued attributes** in the same relation.
- The attributes are **independent**, but both depend on the same key.

💡 Example:

```
EMP(Ename, Pname, Dname)
```

- An employee (`Ename`) **works on multiple projects (Pname)**.
- The same employee also **has multiple dependents (Dname)**.
- But **projects and dependents are independent** — a dependent doesn't depend on which project the employee works on.

🧠 2 The Problem

To represent this properly, for every **combination** of `(Ename, Pname)` and `(Ename, Dname)`,

we need to store **all possible pairs** of (project, dependent) for each employee.

So, if:

- Smith works on {X, Y}, and
- Smith has dependents {John, Anna},

Then the table must include **4 tuples**:

Ename	Pname	Dname
Smith	X	John
Smith	X	Anna

Ename	Pname	Dname
Smith	Y	John
Smith	Y	Anna

That's a **Cartesian product** of projects × dependents per employee.

This is **redundant** and can cause **insertion/deletion anomalies**, even though no functional dependencies exist.



3 Defining a Multivalued Dependency (MVD)

◆ Formal Definition

A **multivalued dependency** $X \twoheadrightarrow Y$ on a relation schema R means:

If two tuples t_1 and t_2 exist in R such that $t_1[X] = t_2[X]$, then R must also contain two other tuples t_3 and t_4 such that:

- $t_3[X] = t_4[X] = t_1[X] = t_2[X]$
- $t_3[Y] = t_1[Y]$ and $t_4[Y] = t_2[Y]$
- $t_3[Z] = t_2[Z]$ and $t_4[Z] = t_1[Z]$, where $Z = R - (X \cup Y)$

This ensures that all combinations of the Y-values and Z-values for the same X must exist.

💡 In EMP(Ename, Pname, Dname):

- $X = \text{Ename}$
- $Y = \text{Pname}$
- $Z = \text{Dname}$

So:

Ename $\twoheadrightarrow$ Pname
Ename $\twoheadrightarrow$ Dname

Each Ename's projects and dependents must freely combine.



4 Trivial and Nontrivial MVDs

Type	Condition	Example
Trivial MVD	(a) $Y \subseteq X$, or (b) $X \cup Y = R$	$\text{Ename} \twoheadrightarrow \text{Pname}$ in <code>EMP_PROJECTS</code>
Nontrivial MVD	Neither (a) nor (b)	$\text{Ename} \twoheadrightarrow \text{Pname}$ in <code>EMP(Ename, Pname, Dname)</code>

Only **nontrivial MVDs** cause redundancy and need normalization.

5 Fourth Normal Form (4NF)

◆ Definition:

A relation schema **R** is in **4NF** with respect to a set of dependencies **F** (if **F** includes both FDs and MVDs)

if and only if, for every **nontrivial MVD** $X \twoheadrightarrow Y$ in F^+ ,
X is a superkey of **R**.

6 Applying to the Example

EMP(Ename, Pname, Dname)

has:

```
Ename → Pname
Ename → Dname
```

But **Ename** is **not a superkey**, so **EMP** is **not in 4NF**.

Decomposition into 4NF Relations

We decompose as:

Relation	Attributes	Reason
EMP_PROJECTS	(Ename, Pname)	MVD $\text{Ename} \twoheadrightarrow \text{Pname}$ becomes trivial
EMP_DEPENDENTS	(Ename, Dname)	MVD $\text{Ename} \twoheadrightarrow \text{Dname}$ becomes trivial

Now both are in **4NF** — no redundancy, no spurious tuples.

7 Observations

- **All-key relations** (where the only key is all attributes) are always in BCNF, but *may not be in 4NF* if MVDs exist.
- **4NF** removes redundancy from **independent multi-valued facts**.
- After decomposition, **each MVD becomes trivial**.
- **No FDs** are lost in this case, because none existed.

8 Summary Table

Normal Form	Based On	Removes	Example Problem Fixed
1NF	Atomic values	Repeating groups	Sets in attributes
2NF	Functional Dependency	Partial dependency	Attribute depends on part of key
3NF	Functional Dependency	Transitive dependency	Non-key attribute depends on non-key
BCNF	Functional Dependency	All non-superkey FDs	Redundancy due to FDs
4NF	Multivalued Dependency	Multivalued redundancy	Independent attributes repeated (projects × dependents)

Intuitive Shortcut

Whenever you have one key and two independent multivalued attributes, a multivalued dependency exists, and you need to decompose into 4NF relations.

Would you like me to continue with the **Fifth Normal Form (5NF)** next — which deals with **join dependencies (JDs)**, as shown later in Figure 15.15(c-d)?

08/11/2025, 13:10:22



dependent ??? you mean department

08/11/2025, 13:13:14



Good catch — but in this case, **“dependent”** is actually **correct**, not “department.”

Here’s why 📌

In the example from your text:

“An employee may work on several projects and may have several dependents ...”

- **Dependent** means a **person who depends on the employee**, such as a **child, spouse, or family member** covered by the employee’s benefits.
- It’s a **real-world concept** often used in HR or payroll databases (e.g., *each employee has dependents*).

So:

- **Pname** → project name (projects the employee works on)
- **Dname** → dependent name (people who depend on the employee)

The example is designed to show that:

- Projects and dependents are **independent multivalued attributes** for each employee.
- That’s what causes the **multivalued dependency**:

$Ename \twoheadrightarrow Pname$

$Ename \twoheadrightarrow Dname$

meaning the employee’s projects and dependents are independent sets.

✅ Therefore, “dependent” (not “department”) is correct.

08/11/2025, 13:13:16



15.7 Join Dependencies

and Fifth Normal Form

In our discussion so far, we have pointed out the problematic functional dependencies and showed how they were eliminated by a process of repeated binary decomposition to remove them during the process of normalization to achieve 1NF, 2NF,

3NF and BCNF. These binary decompositions must obey the NJB property from

Section 16.2.4 that we referenced while discussing the decomposition to achieve

BCNF. Achieving 4NF typically involves eliminating MVDs by repeated binary

decompositions as well. However, in some cases there may be no nonadditive join

decomposition of R into two relation schemas, but there may be a nonadditive join

decomposition into more than two relation schemas.

Moreover, there may be no

functional dependency in R that violates any normal form up to BCNF, and there

may be no nontrivial MVD present in R either that violates 4NF. We then resort to

another dependency called the join dependency and, if it is present, carry out a

multiway decomposition into fifth normal form (5NF). It is important to note that

such a dependency is a very peculiar semantic constraint that is very difficult to

detect in practice; therefore, normalization into 5NF is very rarely done in practice.

17F+ refers to the cover of functional dependencies F , or all dependencies that are implied by F . This is defined in Section 16.1.

15.8 Summary

535

Definition. A join dependency (JD), denoted by $JD(R_1, R_2, \dots, R_n)$, specified on

relation schema R , specifies a constraint on the states r of R . The constraint

states that every legal state r of R should have a nonadditive join decomposition

into $R_1, R_2, \dots, R_n$. Hence, for every such r we have

$$\pi_{R_1}(r) \Join \pi_{R_2}(r) \Join \dots \Join \pi_{R_n}(r) = r$$

Notice that an MVD is a special case of a JD where $n = 2$. That is, a JD denoted as

$JD(R_1, R_2)$ implies an MVD $(R_1 \bowtie R_2) \twoheadrightarrow (R_1 - R_2)$ (or, by symmetry, $(R_1 \bowtie R_2) \twoheadrightarrow (R_2 - R_1)$).

A join dependency $JD(R_1, R_2, \dots, R_n)$, specified on relation schema R ,

is a trivial JD if one of the relation schemas R_i in $JD(R_1, R_2, \dots, R_n)$ is equal to R .

Such a dependency is called trivial because it has the nonadditive join property for

any relation state r of R and thus does not specify any constraint on R . We can now

define fifth normal form, which is also called project-join normal form.

Definition. A relation schema R is in fifth normal form (5NF) (or project-join

normal form (PJNF)) with respect to a set F of functional, multivalued, and

join dependencies if, for every nontrivial join dependency $JD(R_1, R_2, \dots, R_n)$ in

F^+ (that is, implied by F),¹⁸ every R_i is a superkey of R .

For an example of a JD, consider once again the SUPPLY all-key relation in Figure

15.15(c). Suppose that the following additional constraint always holds: Whenever a

supplier s supplies part p , and a project j uses part p , and the supplier s supplies at

least one part to project j , then supplier s will also be supplying part p to project j .

This constraint can be restated in other ways and specifies a join dependency $JD(R_1,$

R2, R3) among the three projections R1(Sname, Part_name), R2(Sname, Proj_name), and R3(Part_name, Proj_name) of SUPPLY. If this constraint holds, the tuples below the dashed line in Figure 15.15(c) must exist in any legal state of the SUPPLY relation that also contains the tuples above the dashed line. Figure 15.15(d) shows how the SUPPLY relation with the join dependency is decomposed into three relations R1, R2, and R3 that are each in 5NF. Notice that applying a natural join to any two of these relations produces spurious tuples, but applying a natural join to all three together does not. The reader should verify this on the sample relation in Figure 15.15(c) and its projections in Figure 15.15(d). This is because only the JD exists, but no MVDs are specified. Notice, too, that the JD(R1, R2, R3) is specified on all legal relation states, not just on the one shown in Figure 15.15(c).

Discovering JDs in practical databases with hundreds of attributes is next to impossible. It can be done only with a great degree of intuition about the data on the part of the designer. Therefore, the current practice of database design pays scant attention to them.

15.8 Summary

In this chapter we discussed several pitfalls in relational database design using intuitive arguments. We identified informally some of the measures for indicating

18 Again, F^+ refers to the cover of functional dependencies F , or all dependencies that are implied by F .

This is defined in Section 16.1.

536

Chapter 15 Basics of Functional Dependencies and Normalization for Relational Databases

whether a relation schema is good or bad, and provided informal guidelines for a good design. These guidelines are based on doing a careful conceptual design in the ER and EER model, following the mapping procedure in Chapter 9 correctly to map entities and relationships into relations. Proper enforcement of these guidelines and lack of redundancy will avoid the insertion/deletion/update anomalies, and generation of spurious data. We recommended limiting NULL values, which cause problems during SELECT, JOIN, and aggregation operations. Then we presented some formal concepts that allow us to do relational design in a top-down fashion by analyzing relations individually. We defined this process of design by analysis and decomposition by introducing the process of normalization.

We defined the concept of functional dependency, which is the basic tool for analyzing relational schemas, and discussed some of its properties. Functional dependencies specify semantic constraints among the attributes of a relation schema. Next we described the normalization process for achieving good designs by testing relations for undesirable types of problematic functional dependencies. We provided a treatment of successive normalization based on a predefined primary key in each relation, and then relaxed this requirement and provided more general definitions of second normal form (2NF) and third normal form (3NF) that take all candidate keys of a relation into account. We presented examples to illustrate how by using the general definition of 3NF a given relation may be analyzed and decomposed to eventually yield a set of relations in 3NF.

We presented Boyce-Codd normal form (BCNF) and discussed how it is a stronger form of 3NF. We also illustrated how the decomposition of a non-BCNF relation must be done by considering the nonadditive decomposition requirement. Then we introduced the fourth normal form based on multivalued dependencies that typically arise due to mixing independent multivalued attributes into a single relation.

Finally, we introduced the fifth normal form, which is based on join dependency, and which identifies a peculiar constraint that causes a relation to be decomposed into several components so that they always yield the original relation back after a join. In practice, most commercial designs have followed the normal forms up to BCNF. Need for decomposing into 5NF rarely arises in practice, and join dependencies are difficult to detect for most practical situations, making 5NF more of theoretical value.

Chapter 16 presents synthesis as well as decomposition algorithms for relational database design based on functional dependencies. Related to decomposition, we discuss the concepts of nonadditive (or lossless) join and dependency preservation, which are enforced by some of these algorithms. Other topics in Chapter 16 include a more detailed treatment of functional and multivalued dependencies, and other types of dependencies.

08/11/2025, 13:49:12



Excellent — this section is the final step in the **normalization hierarchy**, introducing **Join Dependency (JD)** and **Fifth Normal Form (5NF)**, also called **Project-Join**

Normal Form (PJNF).

Let's break it down clearly 📌

◆ 1. Recap Before 5NF

We have seen:

Normal Form	Based On	Removes
1NF	Atomicity	Repeating groups
2NF	Partial Functional Dependencies	Partial key dependencies
3NF	Transitive Dependencies	Transitive dependencies
BCNF	Superkey-based FDs	Anomalies from non-superkey determinants
4NF	Multivalued Dependencies (MVDs)	Independent multi-valued attributes

So far, all these normal forms used **binary decomposition** — splitting one relation into **two** smaller ones.

◆ 2. Why 5NF (Fifth Normal Form)?

Sometimes:

- There is **no FD** (Functional Dependency),
- **No MVD** (Multivalued Dependency),
- But still, **redundancy exists** because the table can be **safely decomposed into more than two relations** (multiway decomposition).

👉 This is where **Join Dependency (JD)** comes in.

◆ 3. Join Dependency (JD)

A **Join Dependency** represents a condition that a relation **can be reconstructed by joining several of its projections** without adding or losing tuples.

Formally:

$$JD(R_1, R_2, \dots, R_n)$$

means that the relation **R** must always be equal to the join of its projections onto $R_1, R_2, \dots, R_n$.

That is:

$$*(\pi_{R_1}(r), \pi_{R_2}(r), \dots, \pi_{R_n}(r)) = r$$

✅ This ensures a **nonadditive (lossless) join** across all decomposed parts.

◆ 4. MVD as a Special Case of JD

A **Multivalued Dependency (MVD)** is a special case of a **Join Dependency** when **n = 2**.

So:

$$JD(R_1, R_2) \Rightarrow MVD((R_1 \cap R_2) \twoheadrightarrow (R_1 - R_2))$$

Hence, **every MVD is a JD**, but **not every JD is an MVD**.

◆ 5. Definition of Fifth Normal Form (5NF)

A relation **R** is in **5NF (or PJNF)** if:

*For every **nontrivial Join Dependency** $JD(R_1, R_2, \dots, R_n)$ that holds in **R**, each R_i is a **superkey** of **R**.*

💡 “Nontrivial” JD means it is not automatically true (i.e., not just R itself).

◆ 6. Example - The SUPPLY Relation

From the figure in your text:

```
SUPPLY(Sname, Part_name, Proj_name)
```

Interpretation:

- Each **Supplier (Sname)** supplies several **Parts**.
- Each **Supplier** supplies to several **Projects**.
- Each **Project** uses several **Parts**.

Constraint:

If a supplier **s** supplies a part **p**, and a project **j** uses that part, and supplier **s** supplies at least one part to **j**,
then **s** supplies **p** to **j**.

This describes a **Join Dependency** among:

```
R1(Sname, Part_name)
R2(Sname, Proj_name)
R3(Part_name, Proj_name)
```

✅ The decomposition:

```
SUPPLY → R1, R2, R3
```

is **lossless only when all three are joined together** (joining any two creates spurious tuples).

Hence, **SUPPLY is not in 5NF**, but each R_i is in 5NF.

◆ 7. Practical Note

- **5NF** is *rarely needed* in practice because such JD-based redundancy is very uncommon.
- Detecting a JD in large databases is **extremely difficult** — it often requires **deep domain knowledge** rather than algorithmic detection.
- Most real-world designs stop at **BCNF** or **4NF**.

◆ 8. Summary

Concept	Based On	Removes	Example
4NF	Multivalued Dependency	Repetition due to independent multi-valued attributes	EMP(Ename, Pname, Dname)
5NF	Join Dependency	Redundancy due to multiway relationships	SUPPLY(Sname, Part_name, Proj_name)

Would you like me to show the **step-by-step decomposition** of the SUPPLY relation into R_1 , R_2 , and R_3 (with the join check)?

08/11/2025, 13:49:16



De Morgan's Axioms (or De Morgan's Laws) are two fundamental rules in **Boolean algebra** and **set theory** that describe how **negation** (NOT or complement) interacts with **AND/OR** operations (or **union/intersection** in sets).

◆ 1. In Boolean Algebra form

Let A and B be Boolean variables (can be 0 or 1).

Then **De Morgan's laws** are:

$$(1) \quad \overline{A + B} = \overline{A} \cdot \overline{B}$$

$$(2) \quad \overline{A \cdot B} = \overline{A} + \overline{B}$$

◆ Meaning:

- The complement of an **OR** is the **AND** of the complements.
- The complement of an **AND** is the **OR** of the complements.

◆ 2. In Set Theory form

Let A and B be sets, and U be the universal set.

Then:

$$(1) \quad (A \cup B)' = A' \cap B'$$

$$(2) \quad (A \cap B)' = A' \cup B'$$

Here:

- $\cup$ = Union (analogous to OR)
- $\cap$ = Intersection (analogous to AND)
- $'$ = Complement (NOT)

◆ 3. In Logic (Propositional) form

For logical statements P and Q :

$$\neg(P \vee Q) \equiv (\neg P) \wedge (\neg Q)$$

$$\neg(P \wedge Q) \equiv (\neg P) \vee (\neg Q)$$

◆ 4. Truth Table Verification

A	B	$A + B$	$\overline{A + B}$	$\overline{A} \cdot \overline{B}$
0	0	0	1	1
0	1	1	0	0
1	0	1	0	0
1	1	1	0	0

✓ Columns for $\overline{A + B}$ and $\overline{A} \cdot \overline{B}$ are identical — proves Law 1.

◆ 5. Practical Applications

- **Digital logic simplification:** converting between NAND and NOR logic.
- **Circuit design:** converting OR-gate networks into equivalent AND-gate networks.
- **Computer programming:** simplifying logical expressions using NOT, AND, OR.
- **Set operations:** complementing unions/intersections.

◆ 6. Example in Logic

$$\neg(A \vee B) = (\neg A) \wedge (\neg B)$$

“Not (A or B)” means “Not A and Not B.”

$$\neg(A \wedge B) = (\neg A) \vee (\neg B)$$

“Not (A and B)” means “Not A or Not B.”

Would you like me to show how these are implemented in **logic circuits** (using NAND/NOR gates)?

08/11/2025, 14:51:19



well... what does it have to do with database design

08/11/2025, 14:51:56



Excellent question — and that's exactly the kind of thinking that leads to good **database design**.

Let's connect the dots 📌

📖 1. A database models real-world data

In any database design, you're essentially **describing a real-world system** in structured form — for example, a university, a library, a store, a social network, etc. But *how* you describe it — what tables you create, what attributes you include, and what relationships exist — depends entirely on **conceptual modeling** using **ER (Entity-Relationship)** principles.

🧩 2. ER model → logical database structure

- **Entities** become **tables** (e.g., `Student` , `Course`).
- **Attributes** become **columns** (e.g., `student_id` , `name` , `age`).
- **Relationships** become **foreign keys or link tables** (e.g., `Enrollment(student_id, course_id)`).

So, learning what entities, attributes, and relationships are — and how to represent them with symbols — is the foundation of good database design.

🔧 3. Why ER modeling matters for design

Before you ever write `CREATE TABLE ...` , you should:

1. Identify entities (things to store info about)
2. Identify attributes (details of those things)
3. Identify relationships (how they're connected)

4. Decide cardinalities (1-to-1, 1-to-many, many-to-many)
5. Convert that into a relational schema

That's exactly what an **ER diagram** does — it's the *blueprint* of your database.

Example

Imagine designing a database for a university.

Concept	ER Model	Database Table
Student	Entity	STUDENT(student_id, name, dept)
Course	Entity	COURSE(course_id, title)
Enrollment	Relationship	ENROLL(student_id, course_id, grade)

So, the “entities, attributes, relationships” you just learned form the **conceptual basis** of your database design. Without them, you'd just be guessing table structures.

In short:

ER modeling → Conceptual design → Logical schema → Physical database

That's how it ties directly to **database design**.

Would you like me to show how the **ER diagram** you just learned can be **converted step by step into actual SQL table definitions**?

08/11/2025, 14:51:58



i mean what does demorgan's axioms have to do with database design

08/11/2025, 14:52:38



Ah — *excellent* clarification. 🙌

You're asking: **why do we even care about De Morgan's laws in database design?**

Let's unpack that carefully — because it's subtle but very important.

🧩 1. De Morgan's laws are about logic

They say:

$$\neg(A \wedge B) = (\neg A) \vee (\neg B)$$

$$\neg(A \vee B) = (\neg A) \wedge (\neg B)$$

In plain English:

- “NOT (A AND B)” means “(NOT A) OR (NOT B)”.
- “NOT (A OR B)” means “(NOT A) AND (NOT B)”.

These laws tell us how to **rewrite logical expressions** that involve NOT, AND, and OR.

🧠 2. SQL and databases rely on logic all the time

Every `WHERE` clause, `JOIN` condition, or query filter is basically a **logical expression**.

Example:

```
SELECT * FROM Student
WHERE NOT (age > 18 AND dept = 'CSE');
```

Using De Morgan's law, this can be rewritten as:

```
SELECT * FROM Student
WHERE age <= 18 OR dept <> 'CSE';
```

Both queries return the **exact same result**, but one might:

- be easier to understand,
- or be optimized differently by the query engine.

So, De Morgan's laws help us **simplify, restructure, and optimize queries**.

⚙️ 3. Where it matters in database design and queries

Area	How De Morgan's laws apply
Query rewriting	Simplify or restructure complex WHERE clauses for readability or performance.
Constraint design	When writing <code>CHECK</code> or <code>WHERE</code> constraints, logical negations may be re-expressed using De Morgan's transformations.
View definitions	When designing database views, logical conditions are often inverted — De Morgan's helps ensure correctness.
Normalization & rule-based systems	Some functional dependency checks or logical implication rules use propositional logic simplification.
Query optimization (DB engine)	Internally, the SQL optimizer uses such logic transformations to find efficient query plans.

💡 4. Example: Filtering customers

Suppose we want customers **who are NOT (from India AND above 30)**.

Using De Morgan:

```
NOT (country = 'India' AND age > 30)
→ country <> 'India' OR age <= 30
```


That makes the condition **clearer** and helps the optimizer evaluate two simpler sub-conditions.

✅ Summary

De Morgan's laws are not used to **design tables** directly — but they are **crucial in the logical foundation** of database design and querying because:

- Databases operate on **Boolean logic**.
 - Every condition in SQL can be simplified using **logical laws**.
 - Understanding them helps you write **correct, efficient, and clear queries**.
-

Would you like me to show a few **SQL query examples** side-by-side where De Morgan's laws are applied to simplify complex conditions?

08/11/2025, 14:52:39